Butterfly-accelerated Gaussian random fields on manifolds

Samuel F. Potter[$,†], Paul Beckman[†], Mike O'Neil[†]

$: Coreform, LLC / Seattle, WA
†: Courant Institute / New York, NY

This talk

- Develop a butterfly factorization for compressing finite element approximations of eigenfunctions of the Laplace-Beltrami operator on surfaces
- Show how this factorization enables fast algorithms for sampling from Gaussian random fields on surfaces and applying matrix-vector products with the GRF covariance matrix

The butterfly factorization

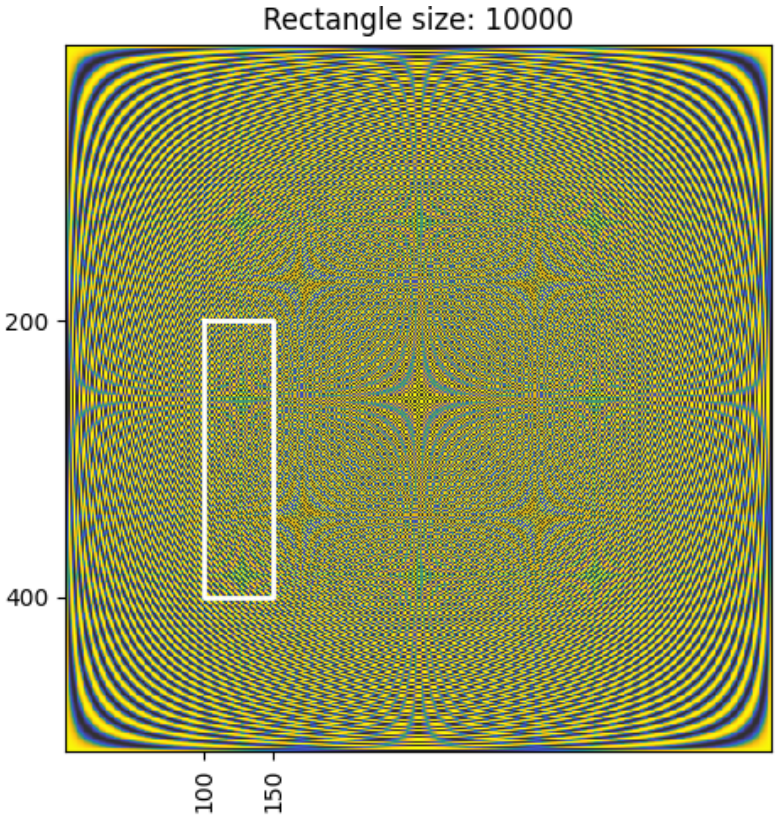
- **1996:** Michielssen and Boag introduce the butterfly factorization for problems in electromagnetic scattering, which turns out to be of broader applicability.
- **2007–10:** O'Neil, Rokhlin, et al. use the butterfly factorization to systematically and rigorously develop fast special function transforms.
- **2010–now:** Extension to higher (3!) dimensions, application to Fourier integral operators, fast direct solvers, etc.

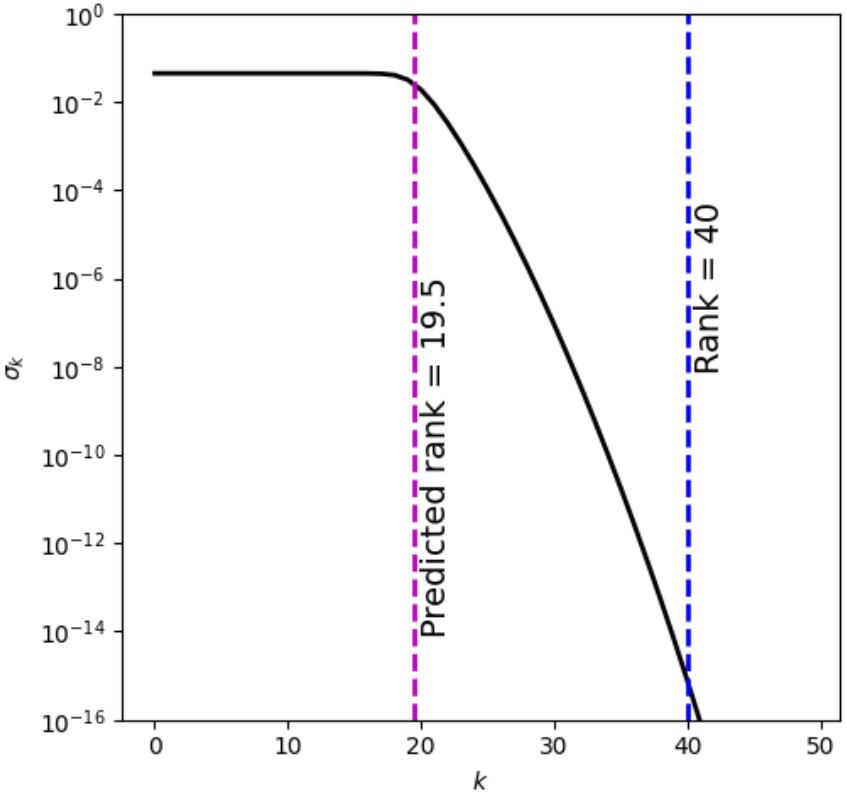
Authors involved: Candes, Demanet, Engquist, Michielssen, Poulson, Ho, Yang, Ying, others.

Example: the discrete Fourier transform

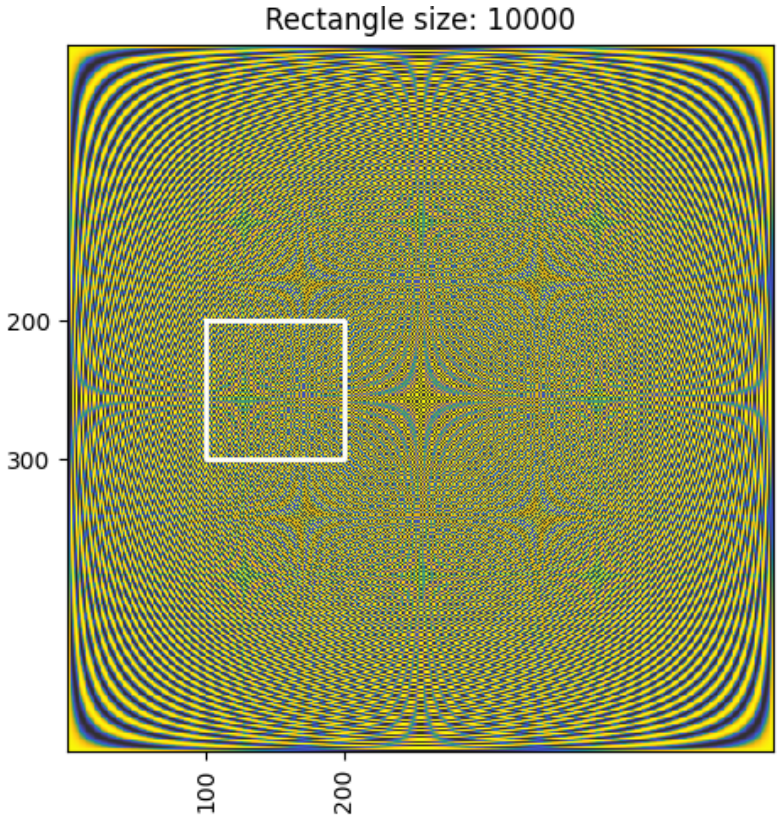
- The DFT: $X \in \mathbb{C}^{N \times N}$, where $X_{kn} = e^{i2\pi kn/N}$
- Easy to prove that the DFT satisfies the butterfly property
- The fast Fourier transform: use algebraic properties of the DFT to apply it in $O(N \log N)$ operations
- Can view the FFT as a sparse matrix factorization of the DFT into $O(\log N)$ sparse factors—so, interpret it as a butterfly factorization
- Can also compute "the" butterfly factorization of the DFT, but very inefficient! (Constant is ~300x worse)

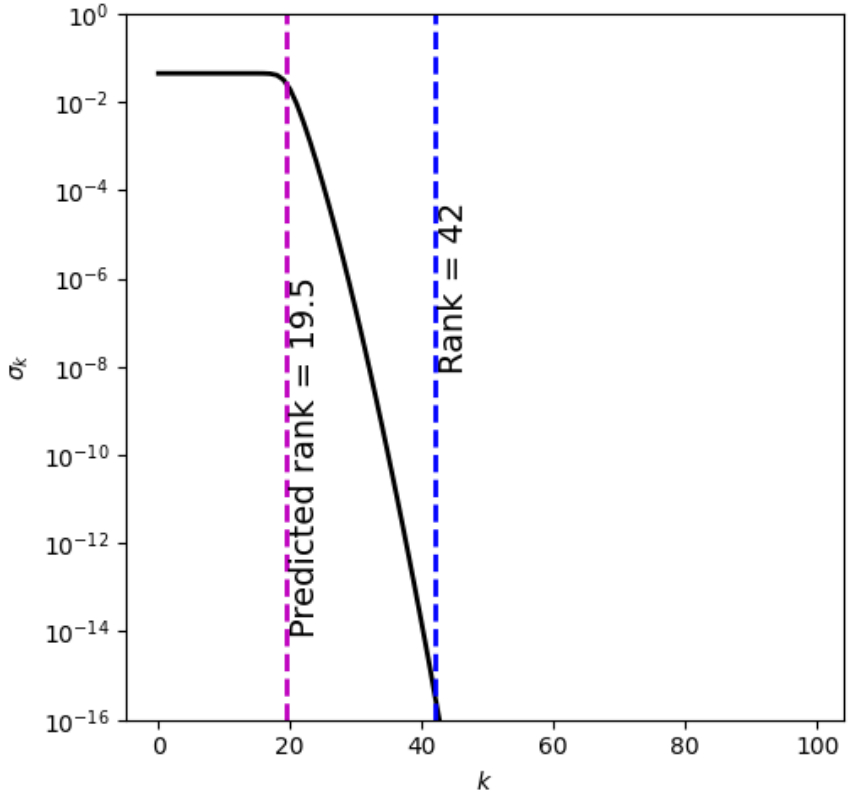
The DFT can be “butterflied”



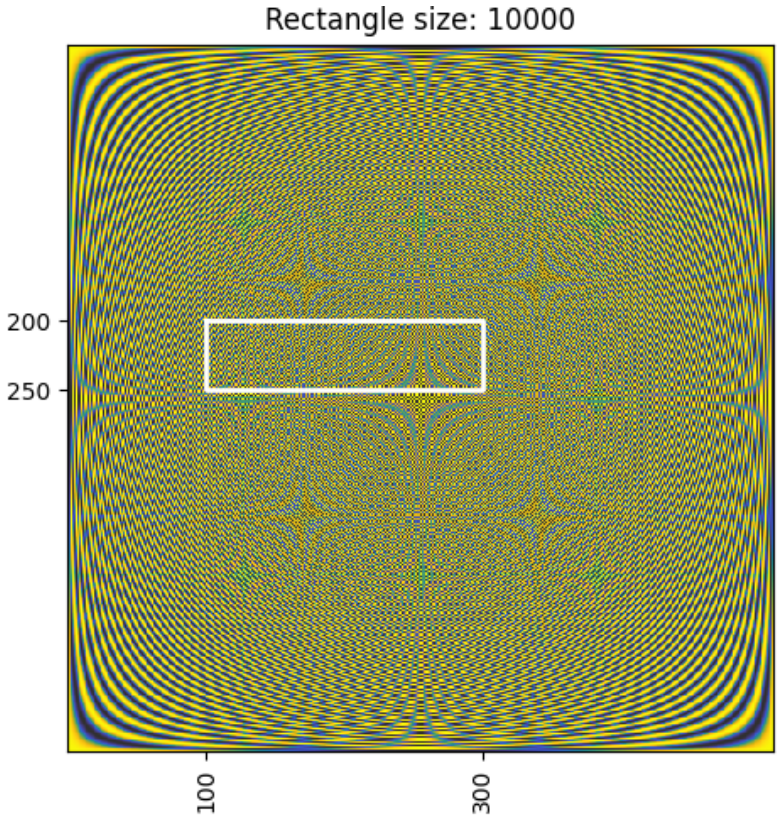


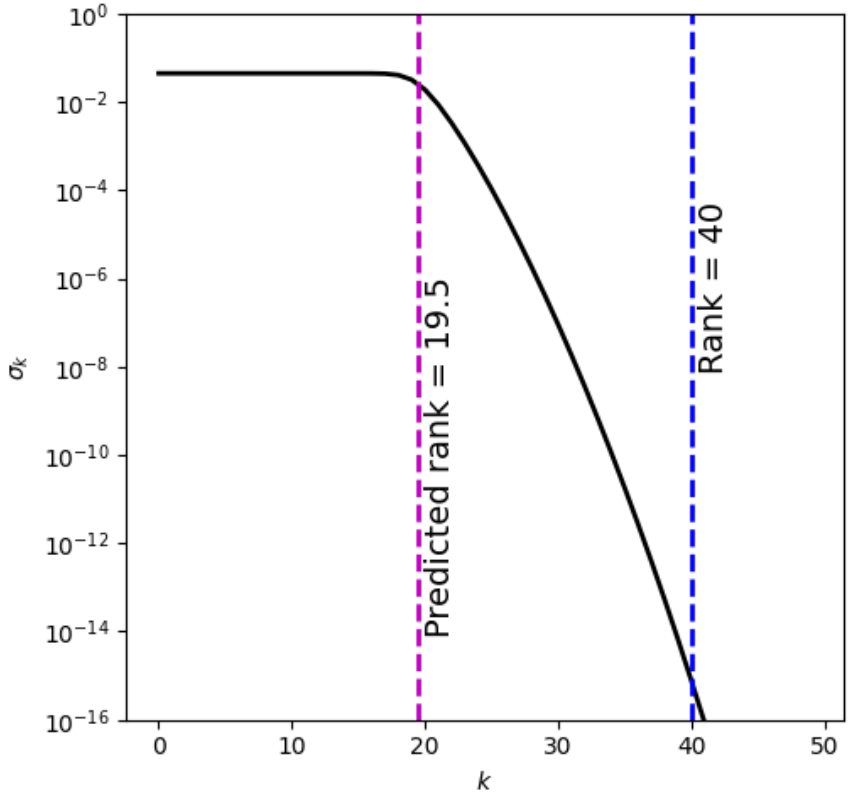
The DFT can be “butterflied”



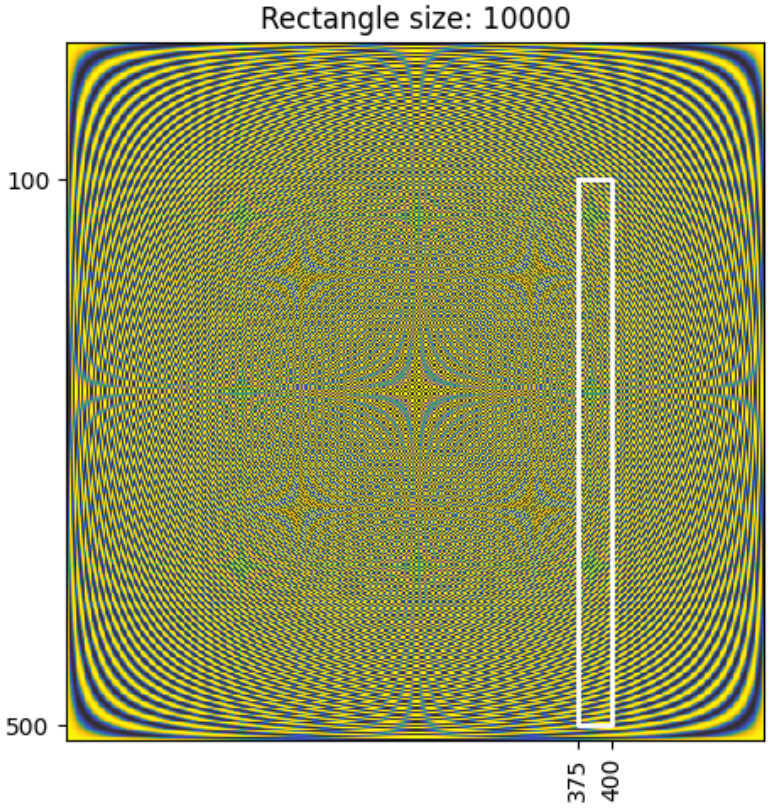


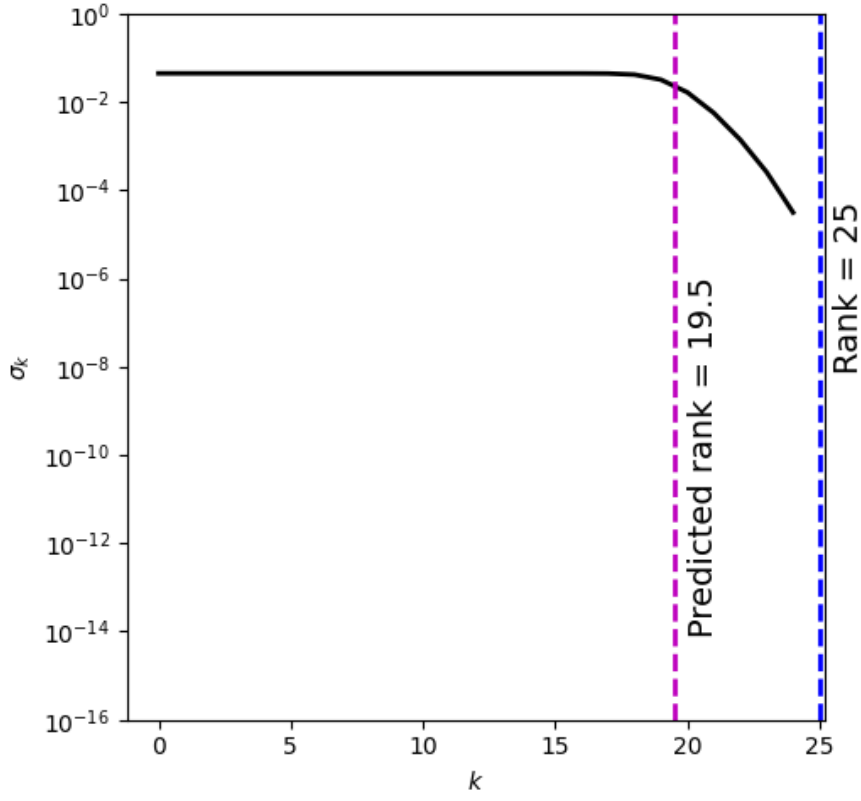
The DFT can be “butterflied”



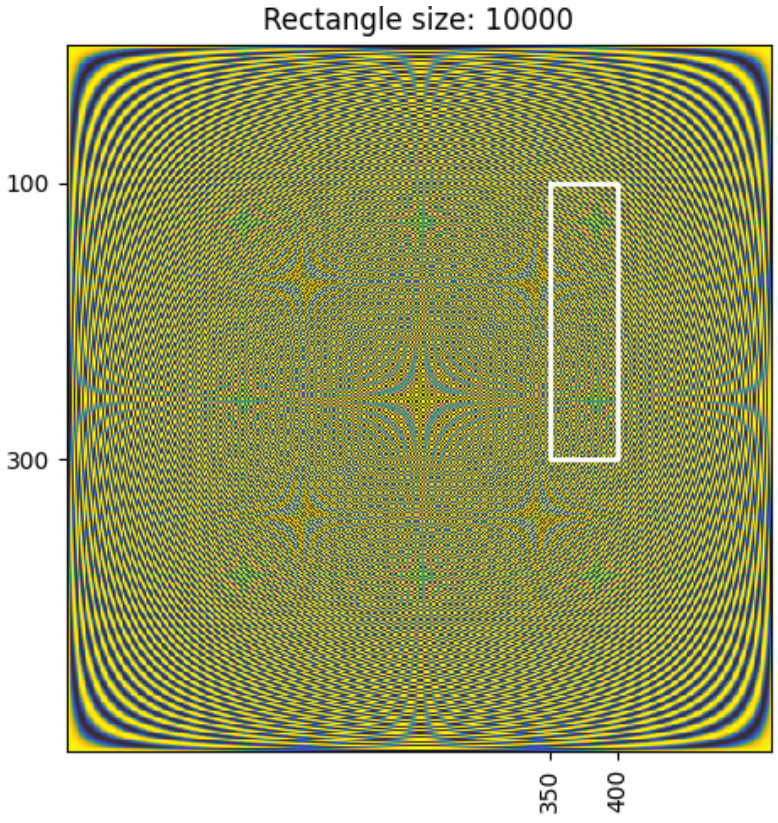


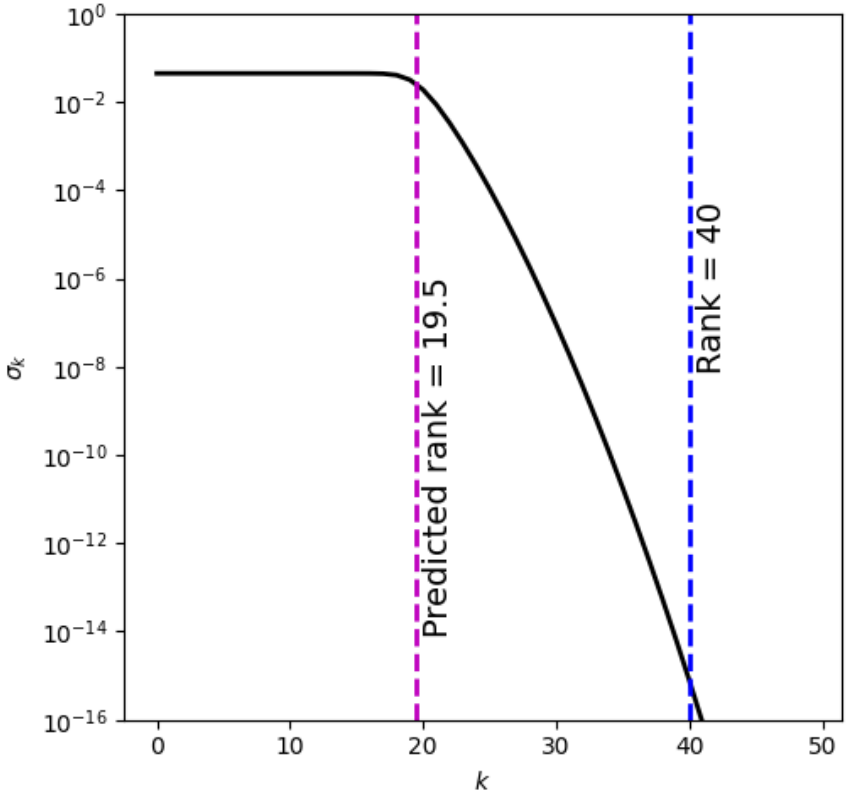
The DFT can be “butterflied”



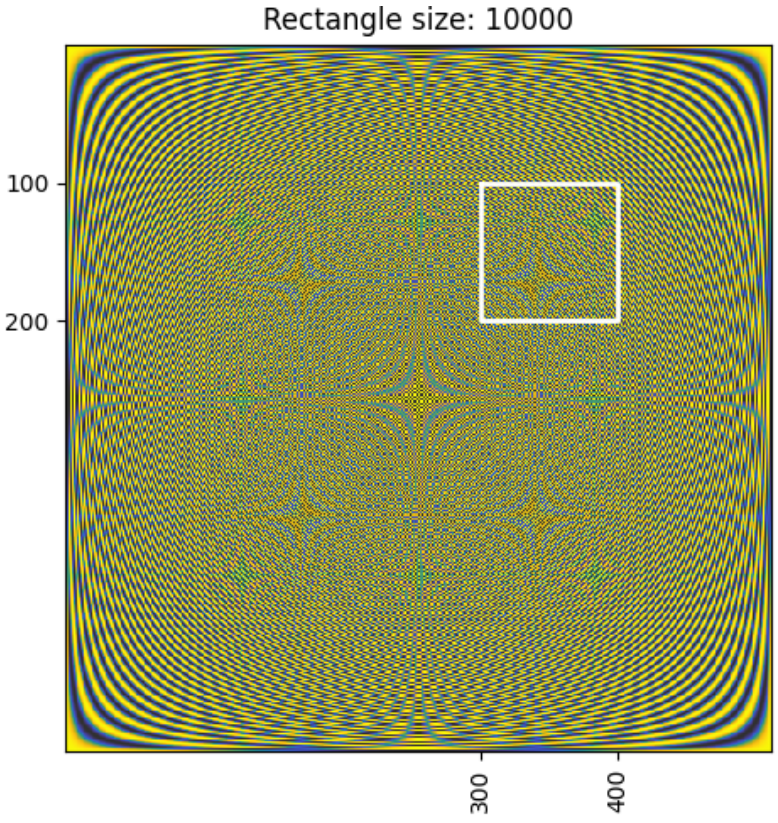


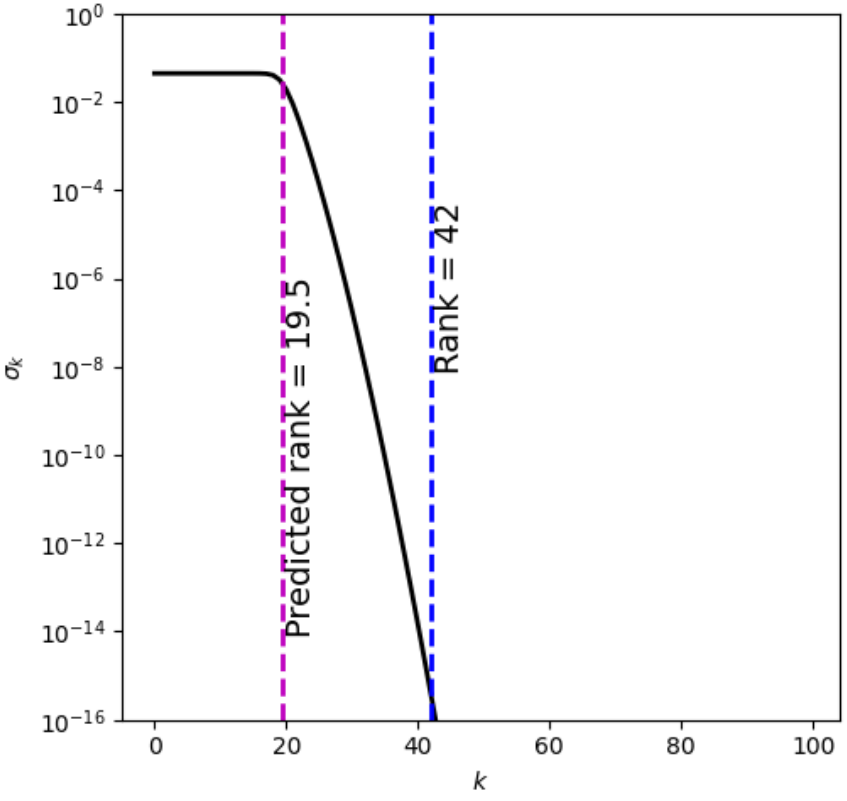
The DFT can be “butterflied”



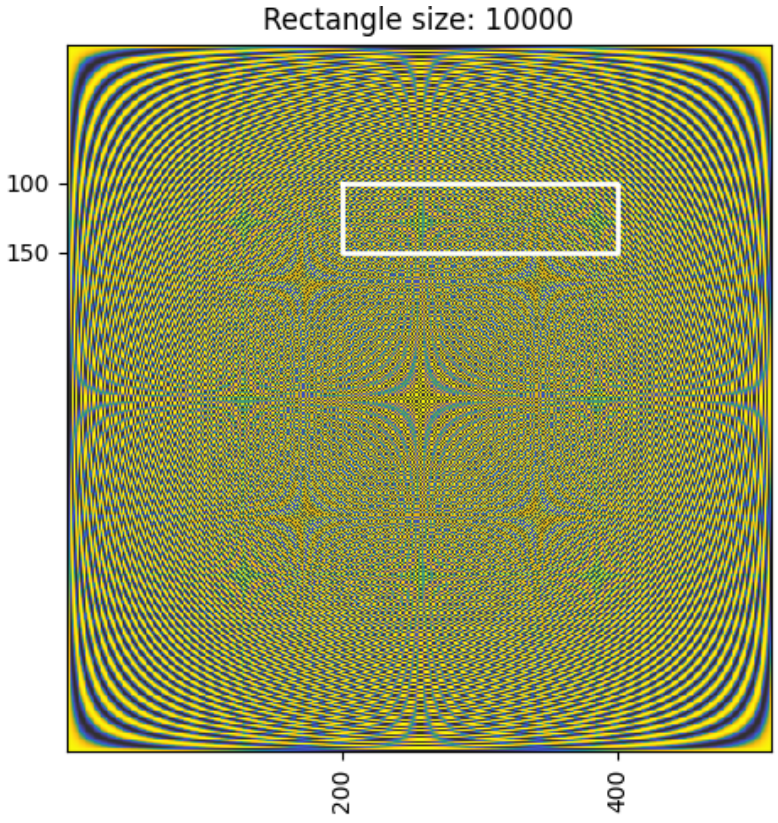


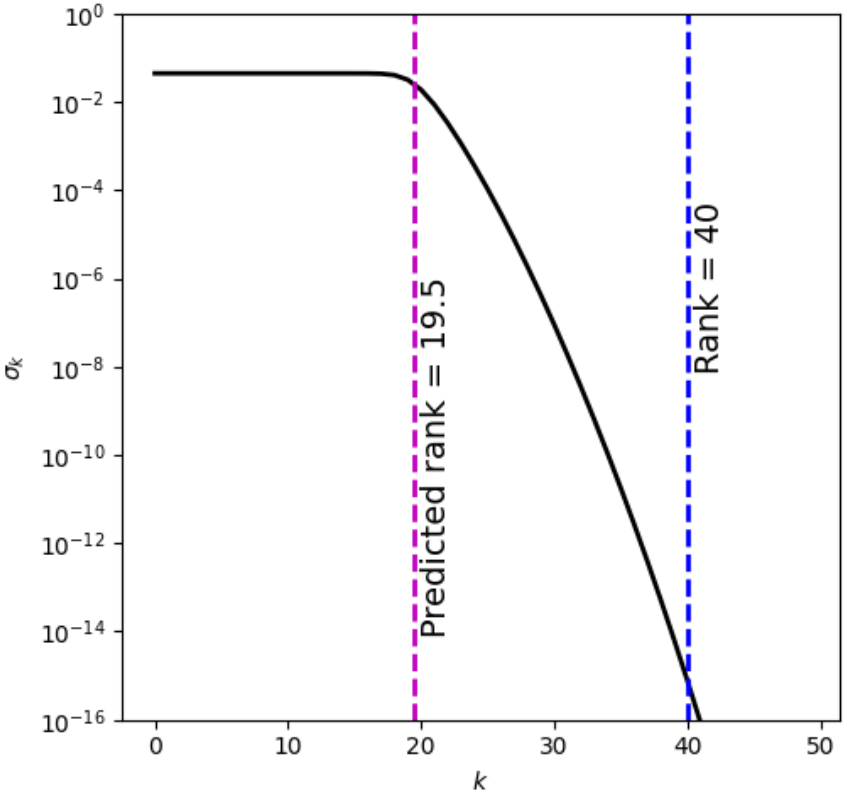
The DFT can be “butterflied”



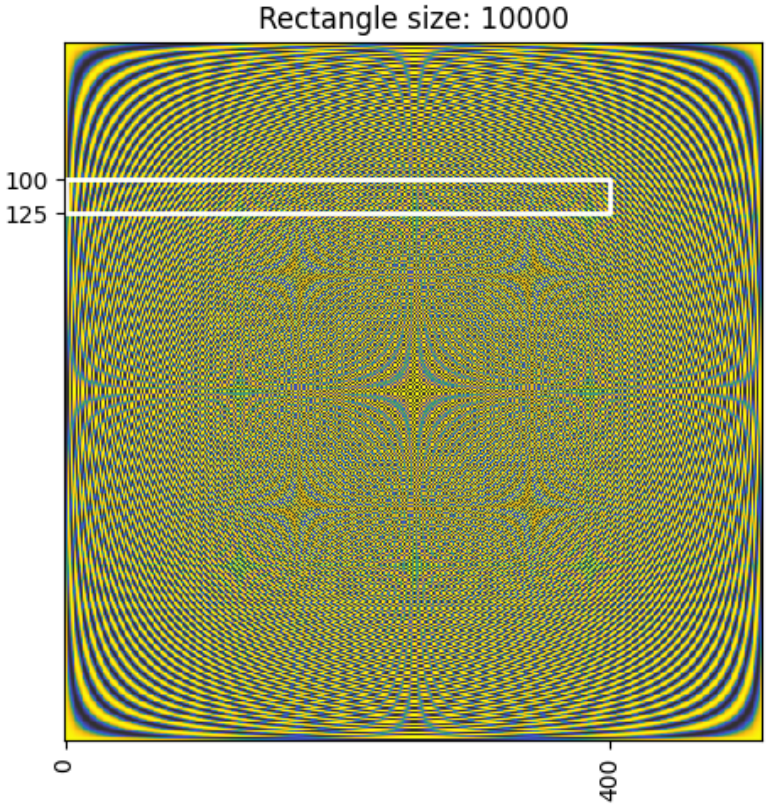


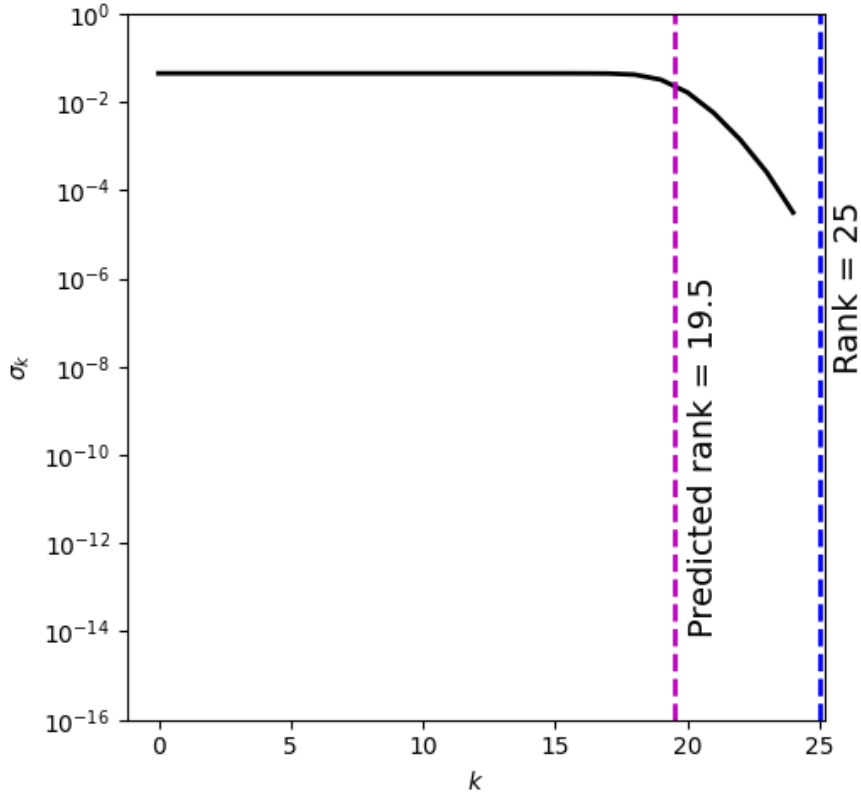
The DFT can be “butterflied”



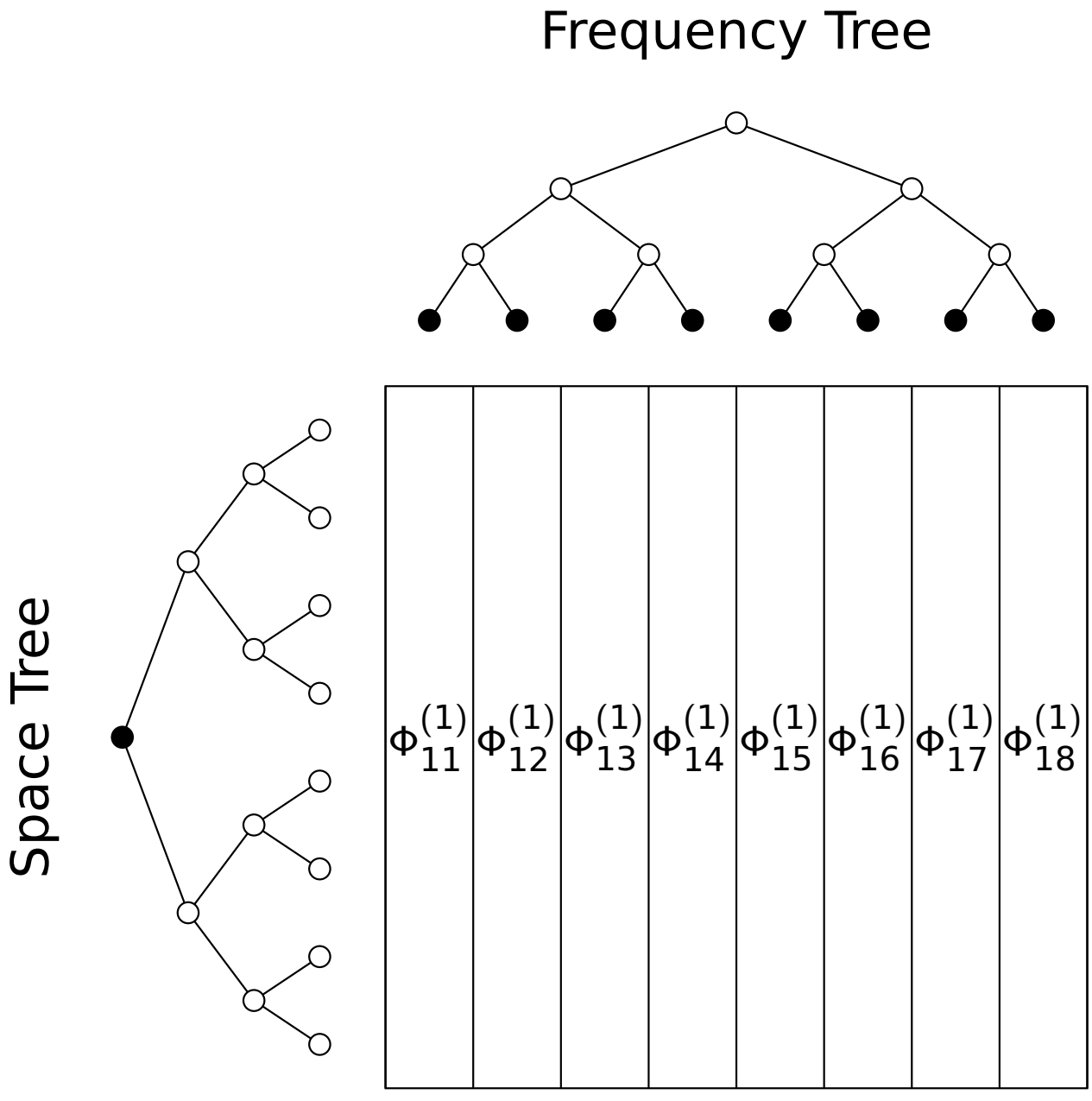


The DFT can be “butterflied”



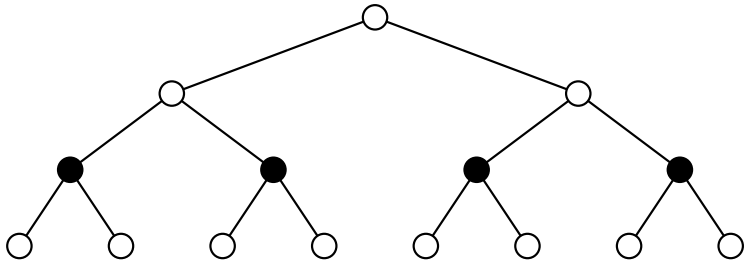


How to compute a column-wise butterfly

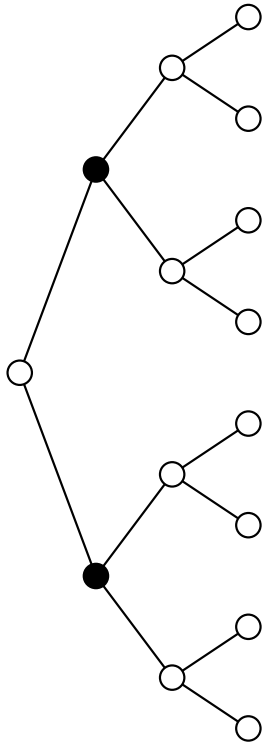


How to compute a column-wise butterfly

Frequency Tree



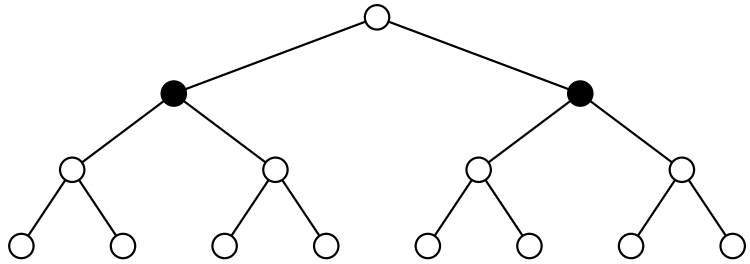
Space Tree



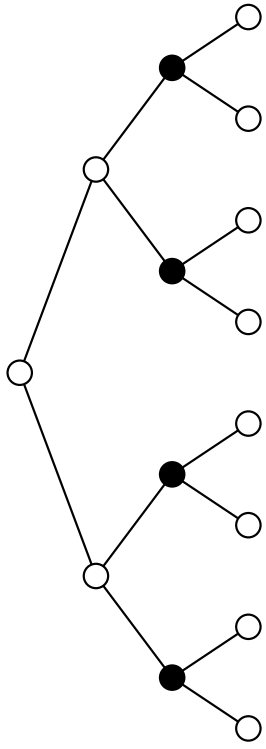
$\Phi_{11}^{(2)}$	$\Phi_{12}^{(2)}$	$\Phi_{13}^{(2)}$	$\Phi_{14}^{(2)}$
$\Phi_{21}^{(2)}$	$\Phi_{22}^{(2)}$	$\Phi_{23}^{(2)}$	$\Phi_{24}^{(2)}$

How to compute a column-wise butterfly

Frequency Tree



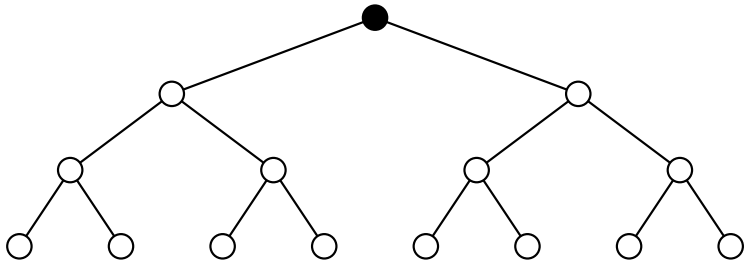
Space Tree



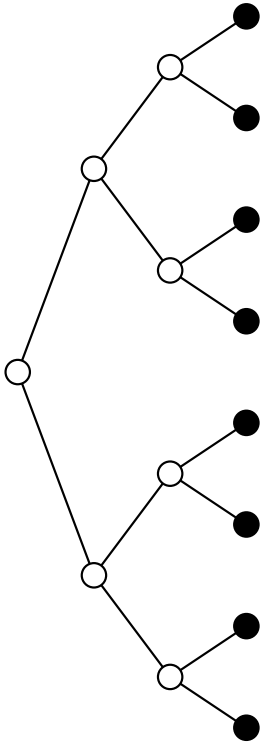
$\Phi_{11}^{(3)}$	$\Phi_{12}^{(3)}$
$\Phi_{21}^{(3)}$	$\Phi_{22}^{(3)}$
$\Phi_{31}^{(3)}$	$\Phi_{32}^{(3)}$
$\Phi_{41}^{(3)}$	$\Phi_{42}^{(3)}$

How to compute a column-wise butterfly

Frequency Tree

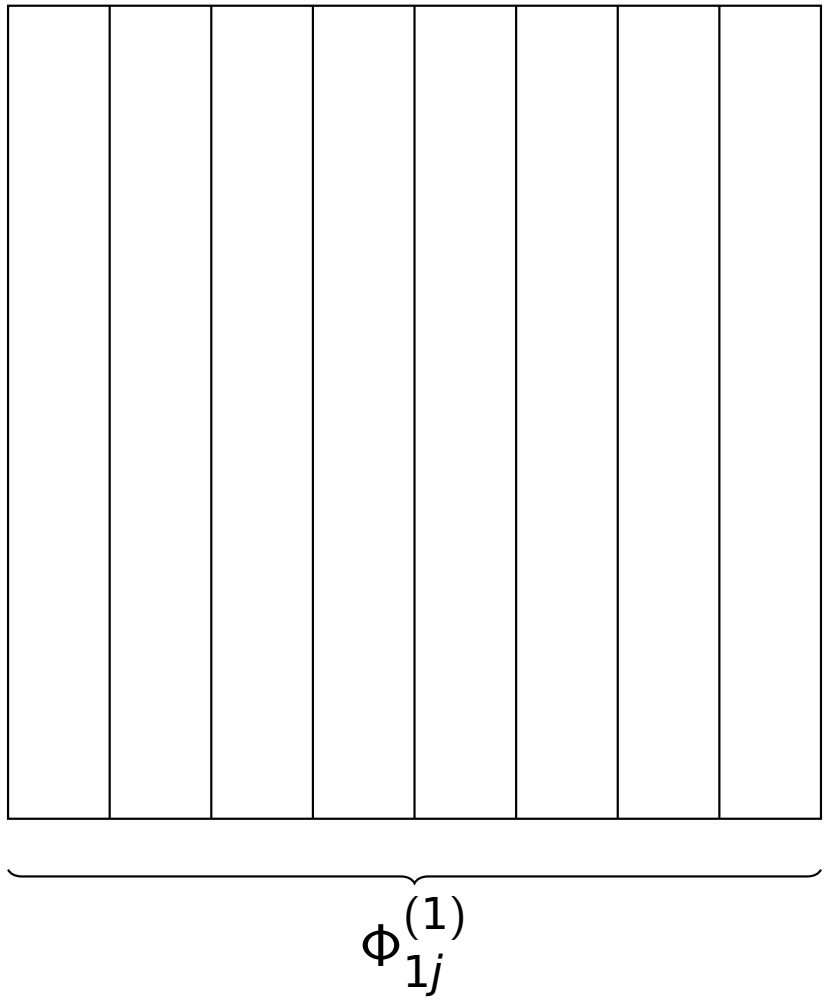


Space Tree

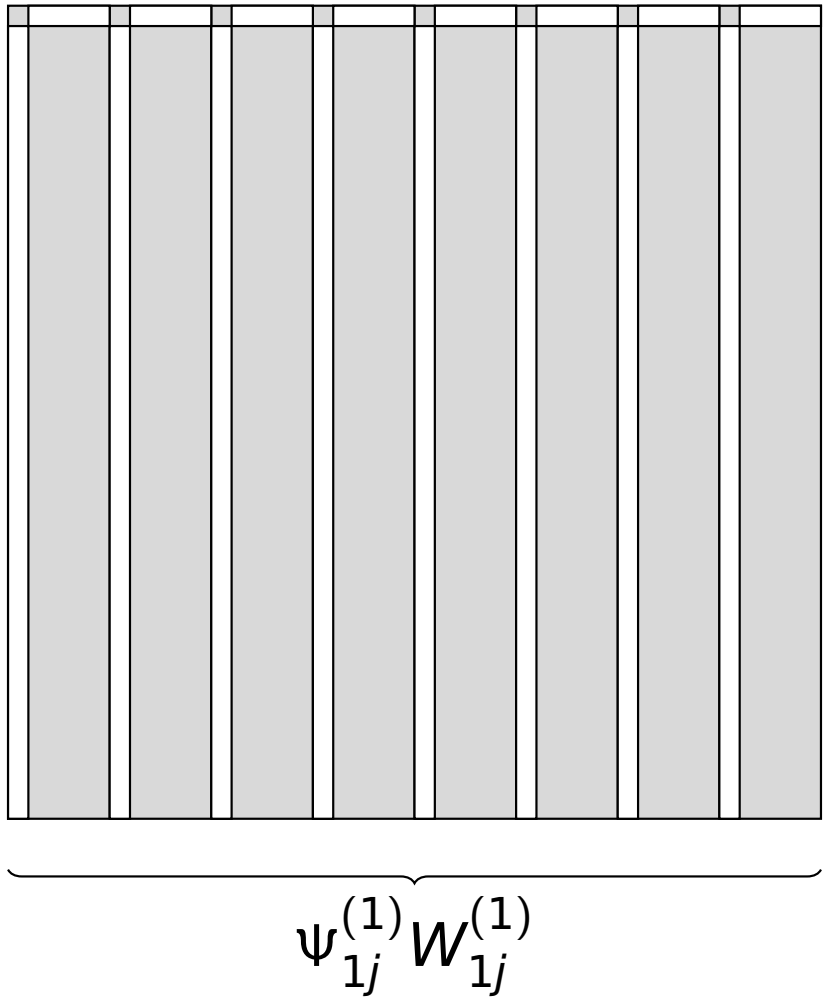


$\Phi_{11}^{(3)}$
$\Phi_{21}^{(3)}$
$\Phi_{31}^{(3)}$
$\Phi_{41}^{(3)}$
$\Phi_{51}^{(3)}$
$\Phi_{61}^{(3)}$
$\Phi_{71}^{(3)}$
$\Phi_{81}^{(3)}$

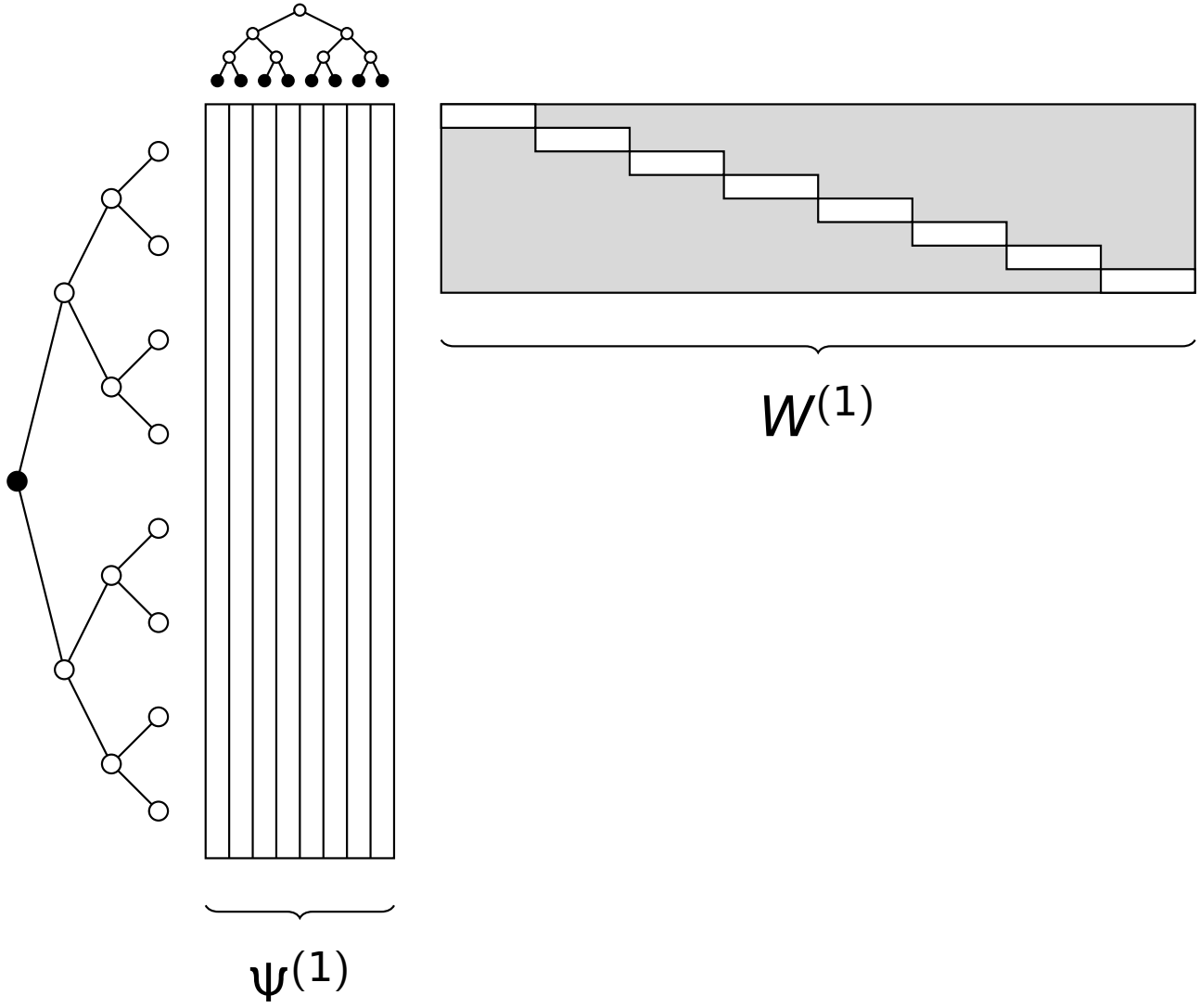
How to compute a column-wise butterfly



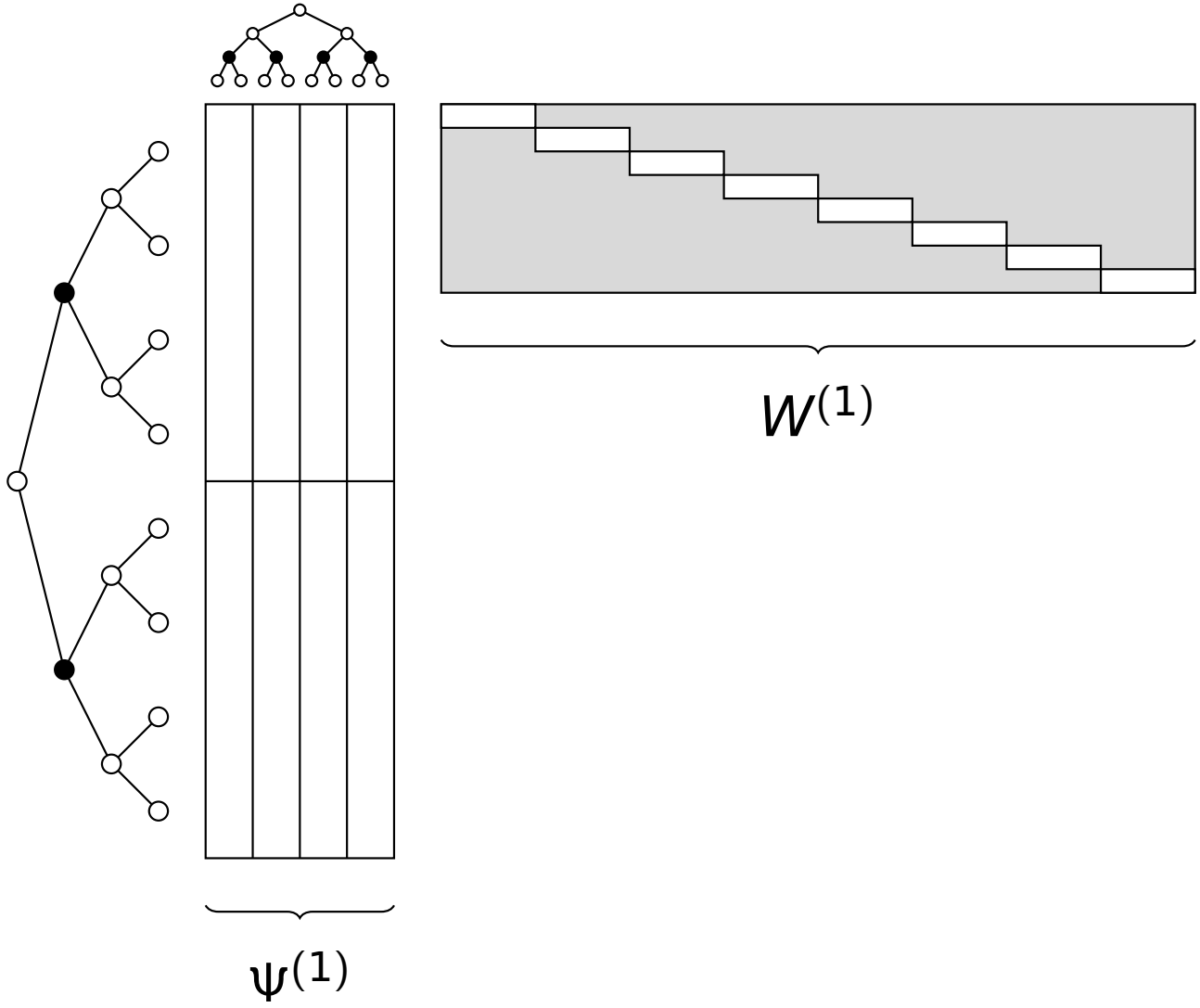
How to compute a column-wise butterfly



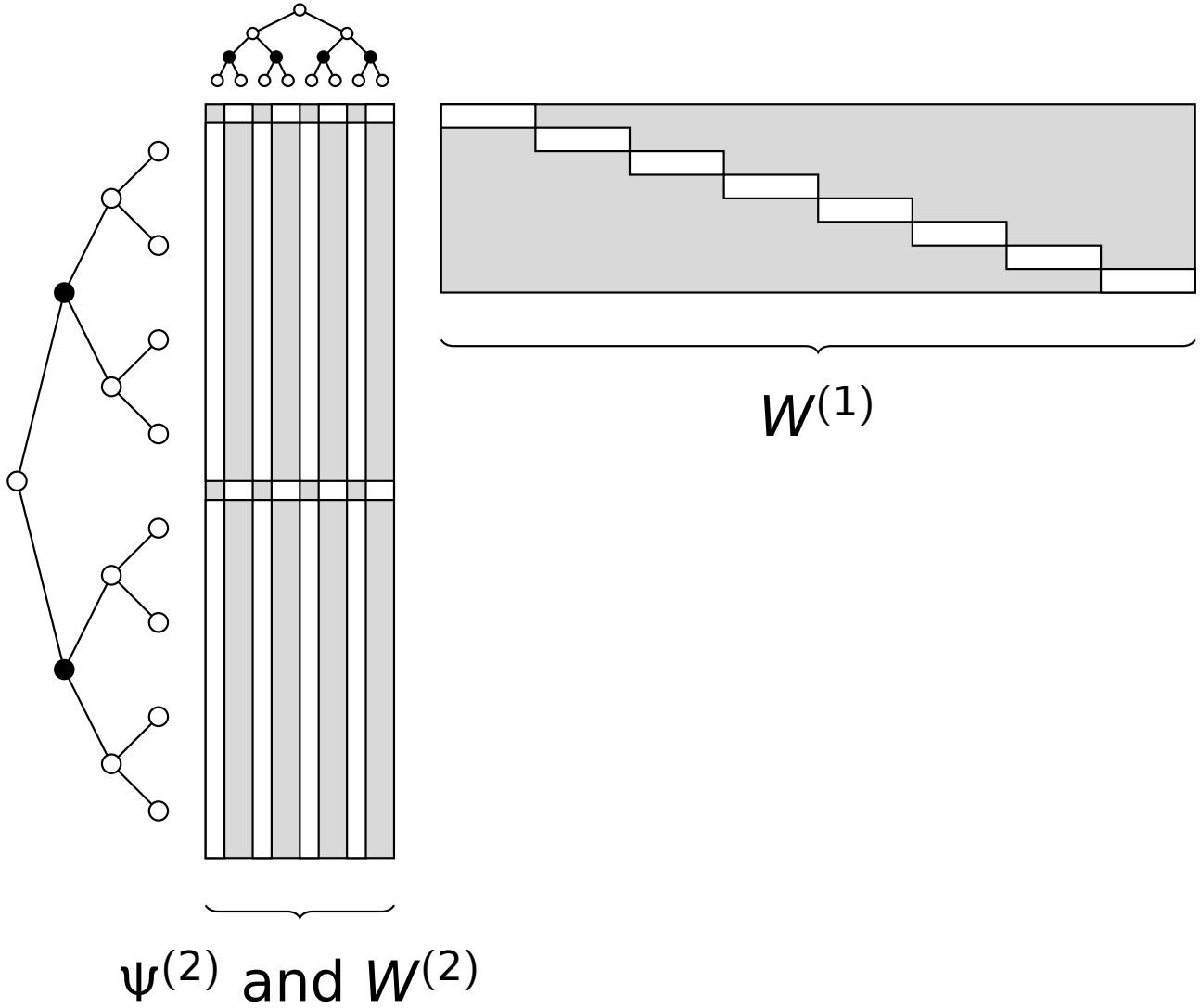
How to compute a column-wise butterfly



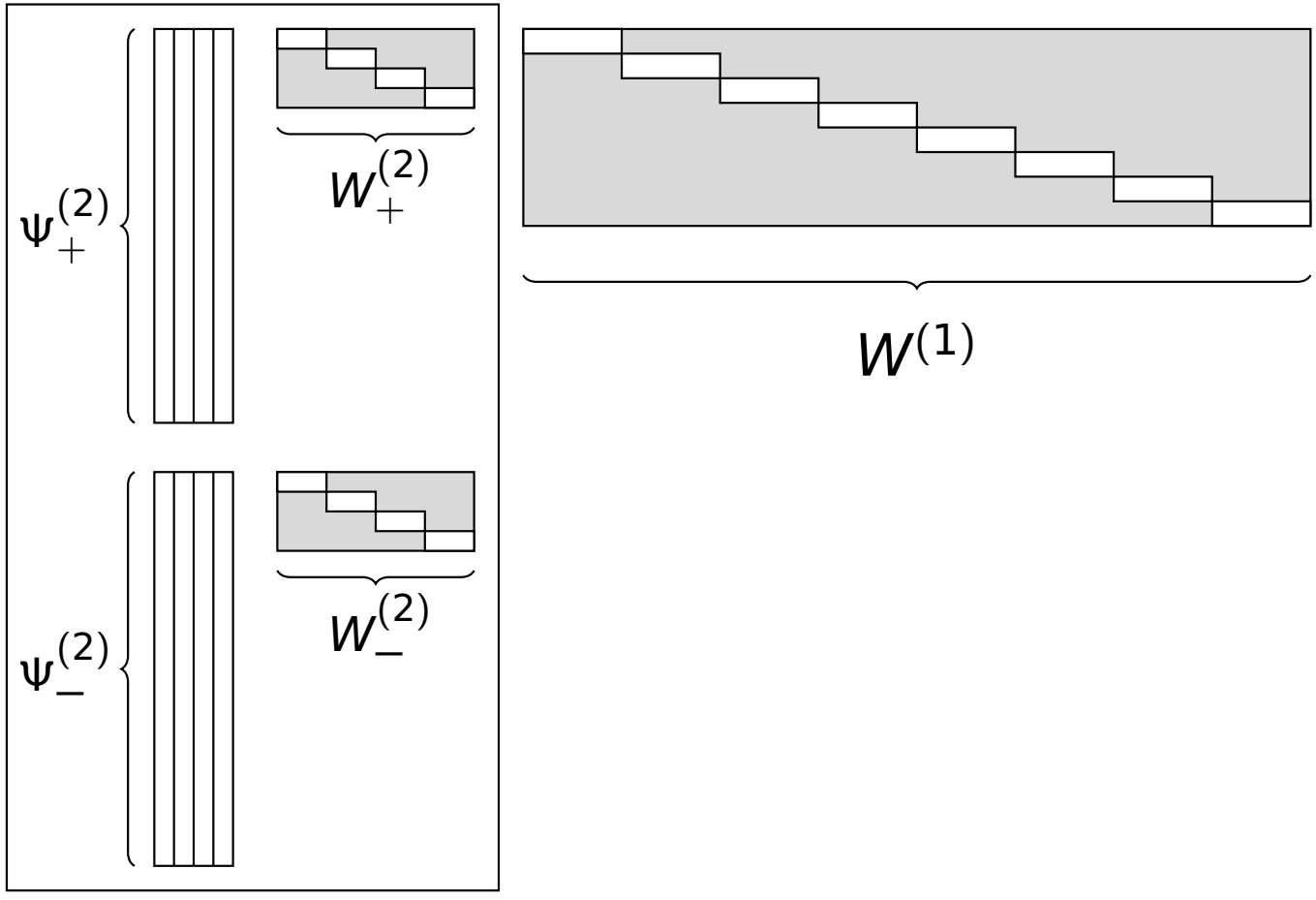
How to compute a column-wise butterfly



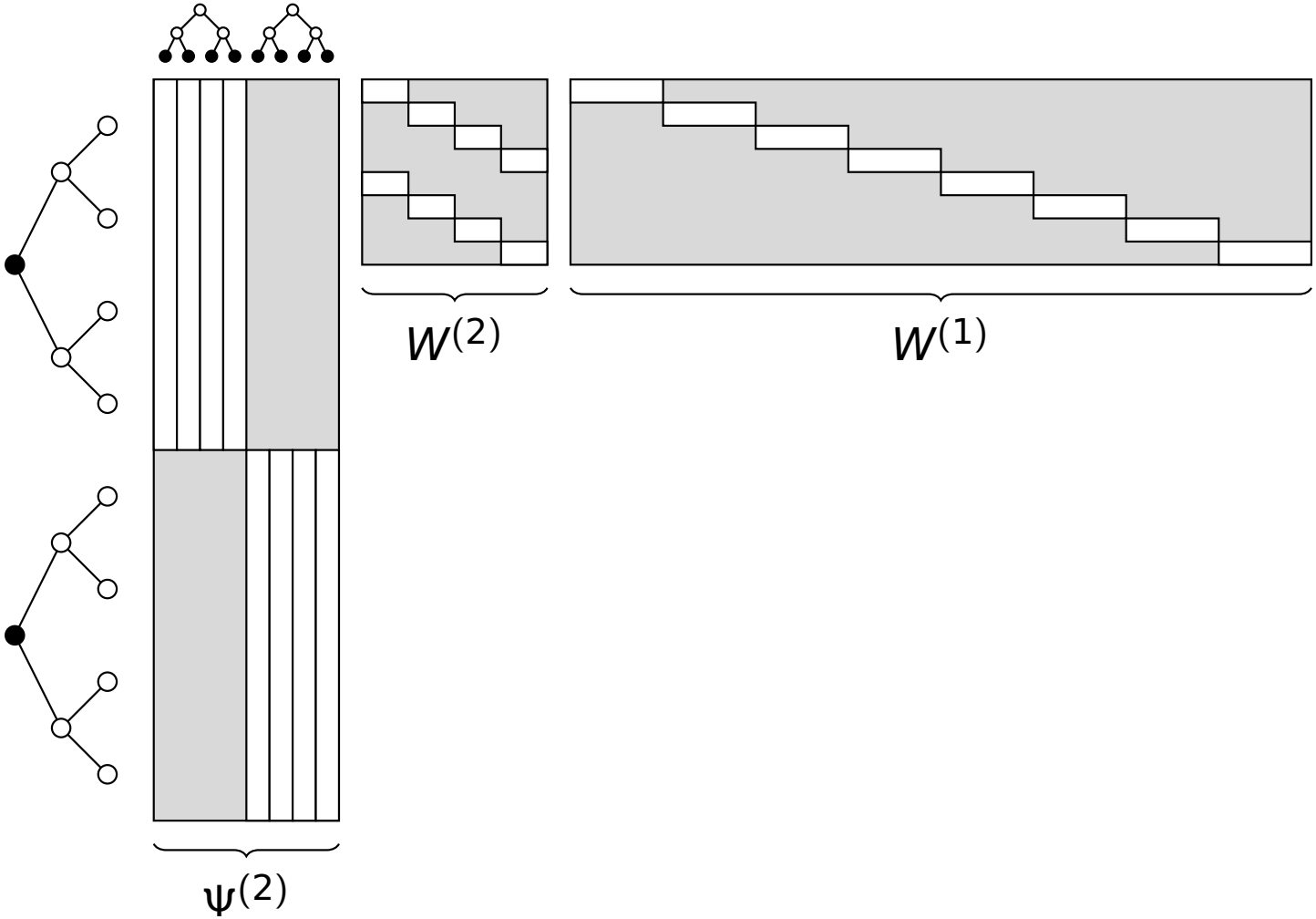
How to compute a column-wise butterfly



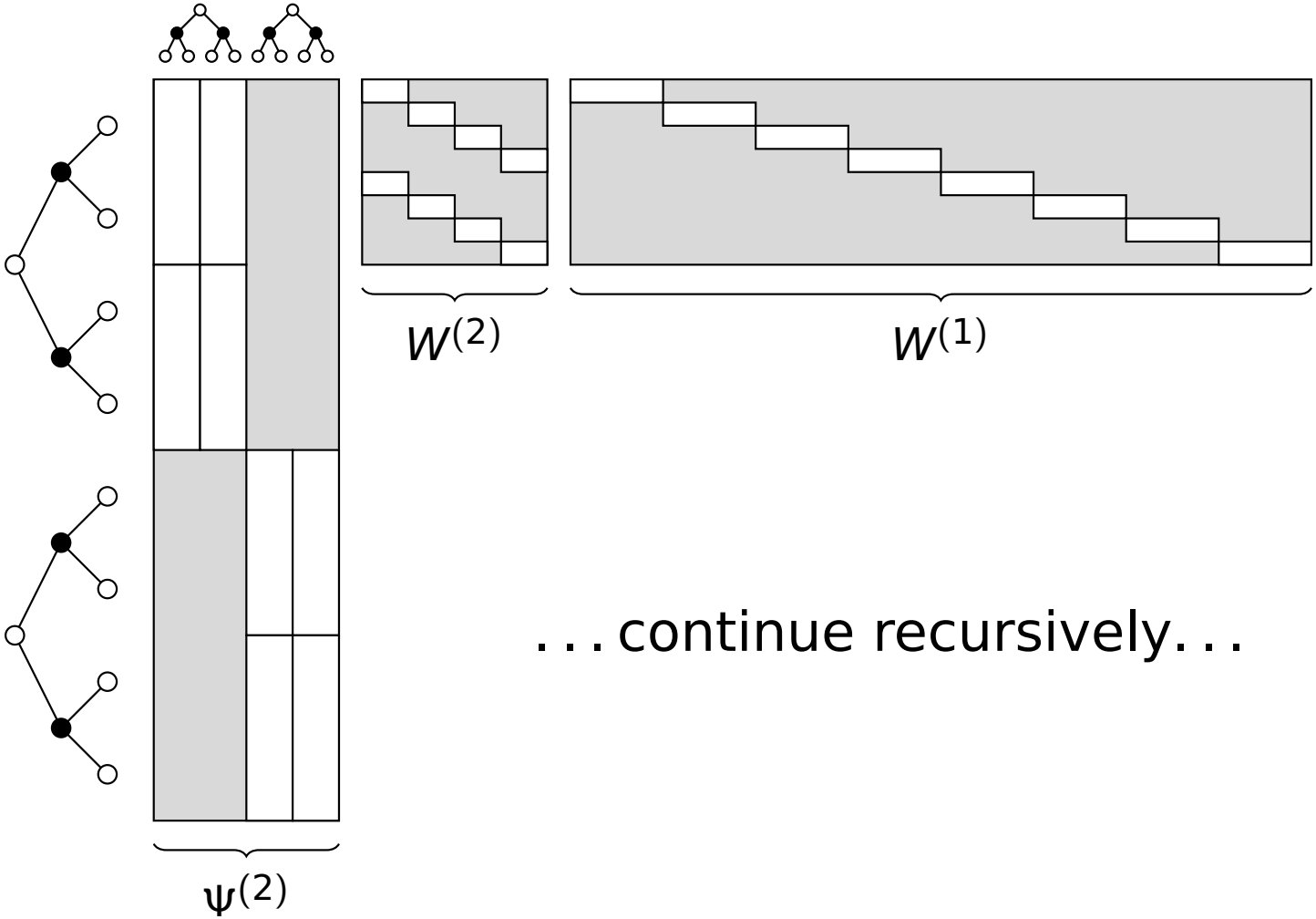
How to compute a column-wise butterfly



How to compute a column-wise butterfly

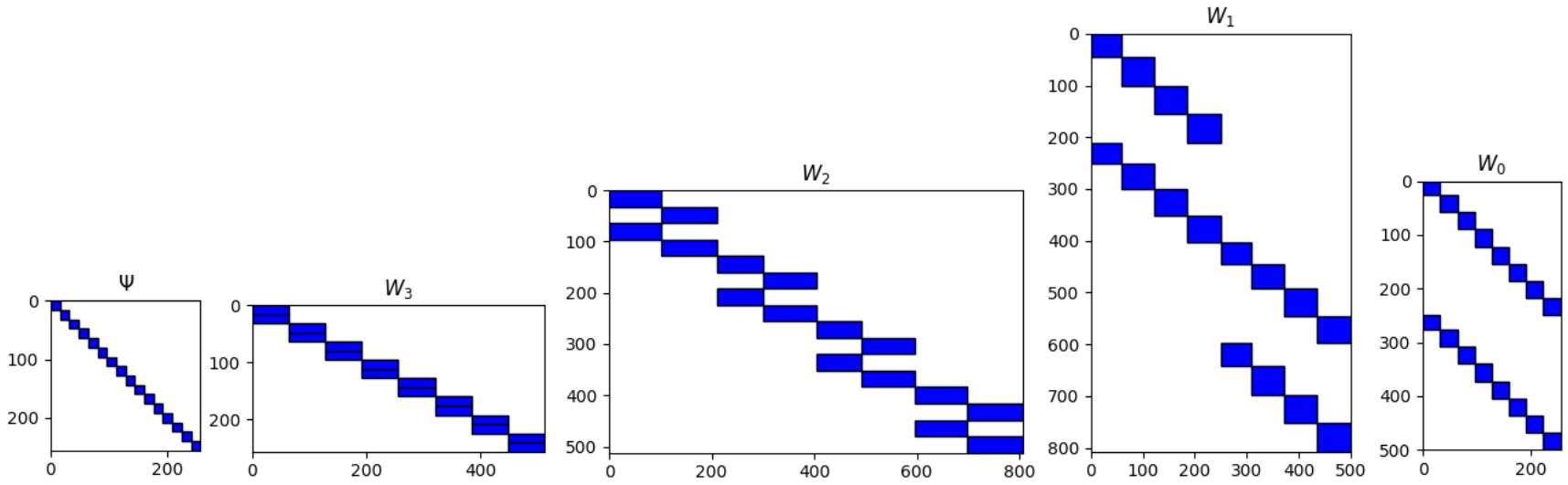


How to compute a column-wise butterfly



. . . continue recursively. . .

Example of a butterfly factorization



Butterfly factorization of 128×128 discrete Legendre transform:

$$\Phi \approx \Psi^{(3)} W^{(3)} W^{(2)} W^{(1)} W^{(0)}, \qquad \Phi_{ij} = P_j(x_i)$$

Bad choice here because of inner factor size blow-up—butterfly compressed matrix is ~2 larger than original. . .

What are “manifold harmonics”?

- Let M be a compact Riemannian manifold. The **Laplace-Beltrami operator** is the Laplacian on M:

$$\Delta_M = \nabla_M \cdot \nabla_M$$

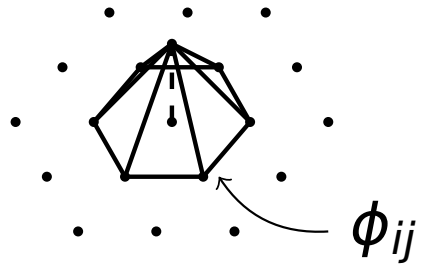
- A **manifold harmonic** is an eigenfunction of the Laplace-Beltrami eigenproblem on M:

$$\Delta_M u = \lambda u, \qquad u \in L^2(M)$$

- Eigenfunctions of Δ_M give orthonormal basis for $L^2(M)$.

Our focus: genus 0 surfaces $S \subseteq \mathbb{R}^3$.

How do we compute manifold harmonics?



- Discretize $S \rightarrow$ tri. mesh T with vertices $V = \{x_i\}_{i \in I}$
- Set up basis of linear finite elements $\{\phi_i\}_{i \in I}$:

$$\phi_i(x_j) = \delta_{ij}, \qquad x_j \in V, \qquad \sum_i u_i \phi_i(x) \in C^0(T)$$

- Leads to generalized eigenvalue problem:

$$Lu = \lambda M u$$

where L and M are stiffness and mass matrices.

How do we compute manifold harmonics?

"Asian dragon" mesh: 56.4K vertices



Source: Stanford 3D Scanning Repository

How do we compute manifold harmonics?

A harmonic ($\lambda \approx 0.848$) on the “Asian dragon” mesh



Source: Stanford 3D Scanning Repository

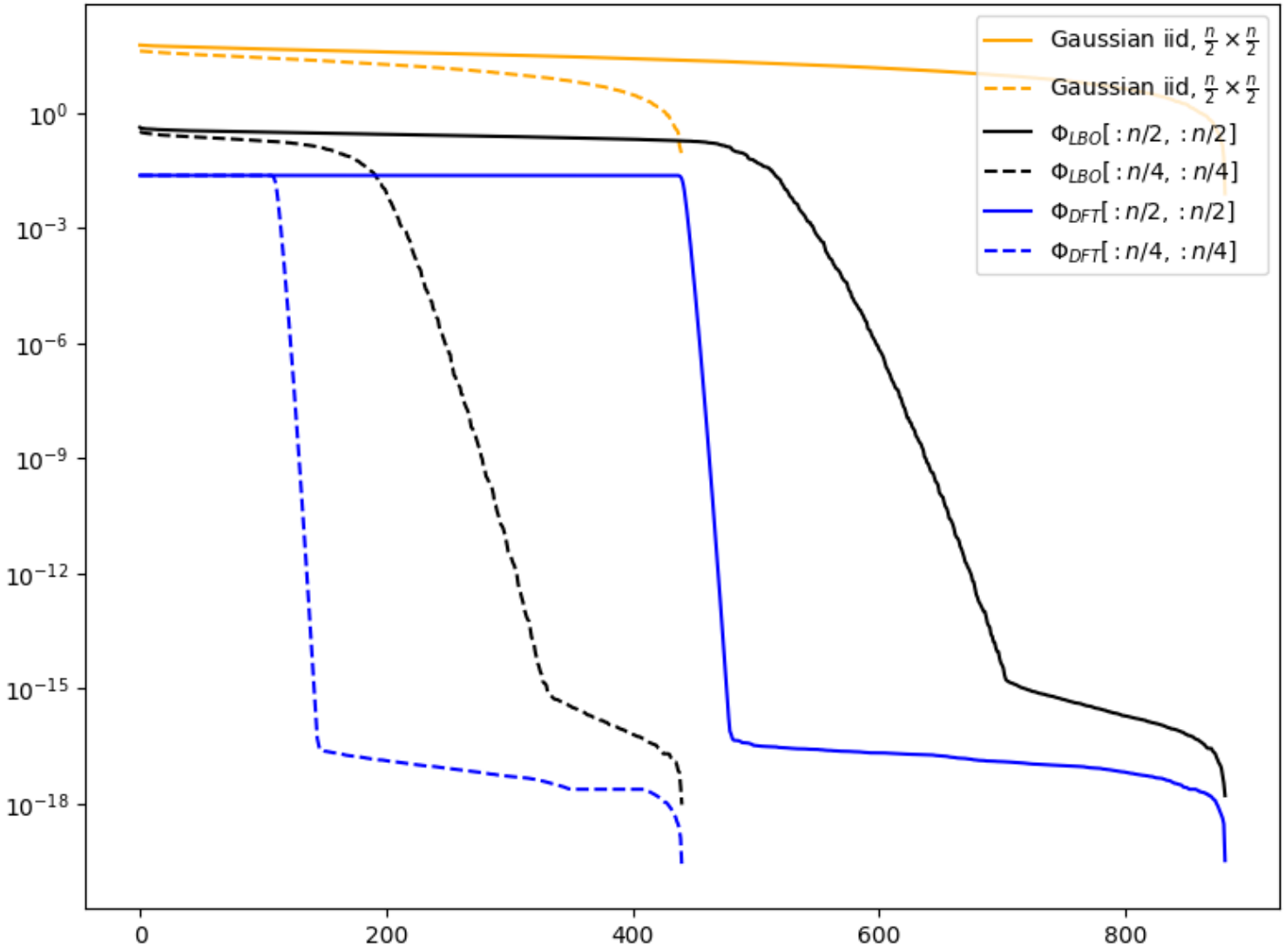
Spatial partitioning

"Asian dragon" with **octree ordering** on vertices



Can we butterfly surface harmonics?

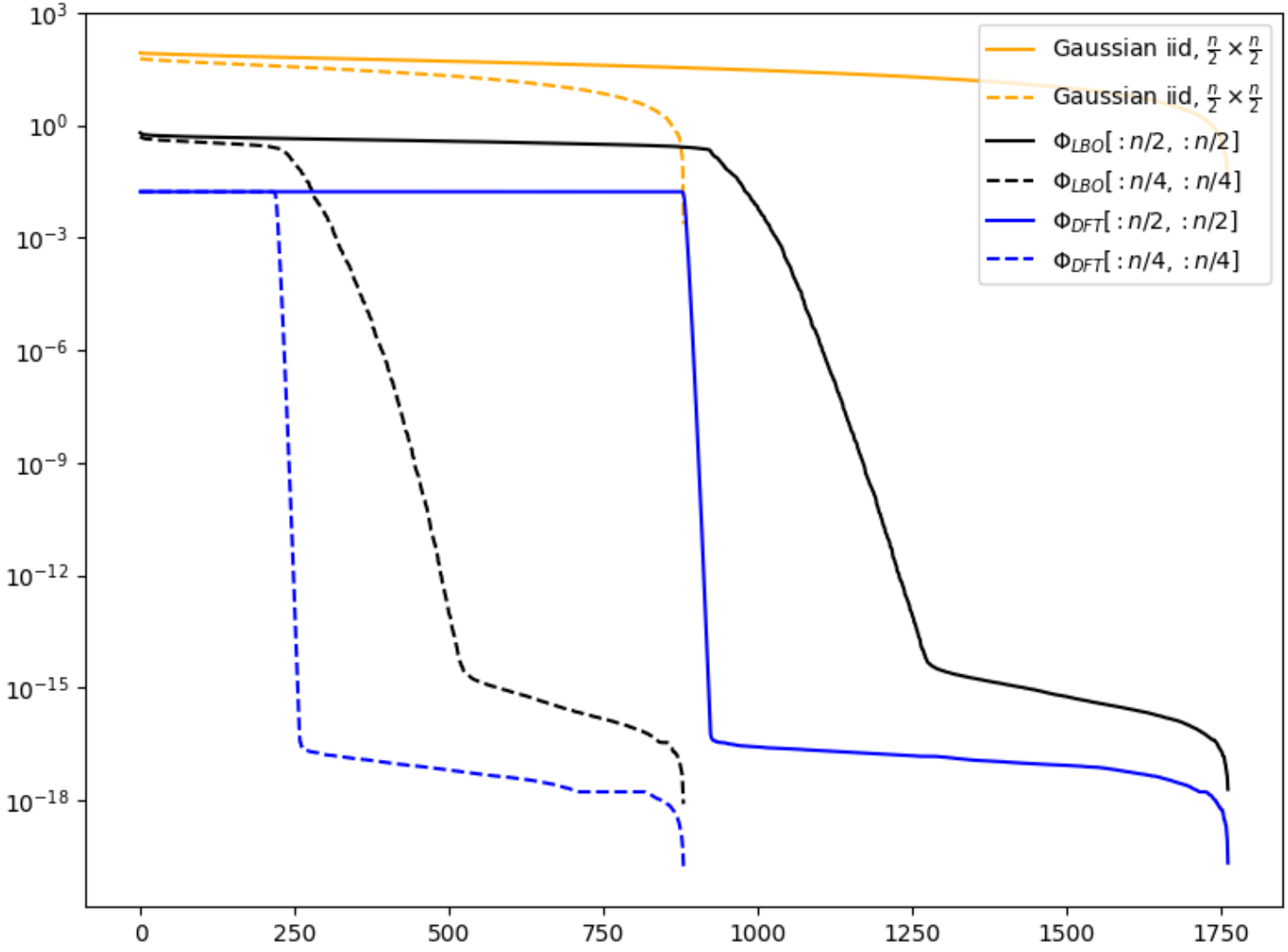
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **1,764** vertices)

Can we butterfly surface harmonics?

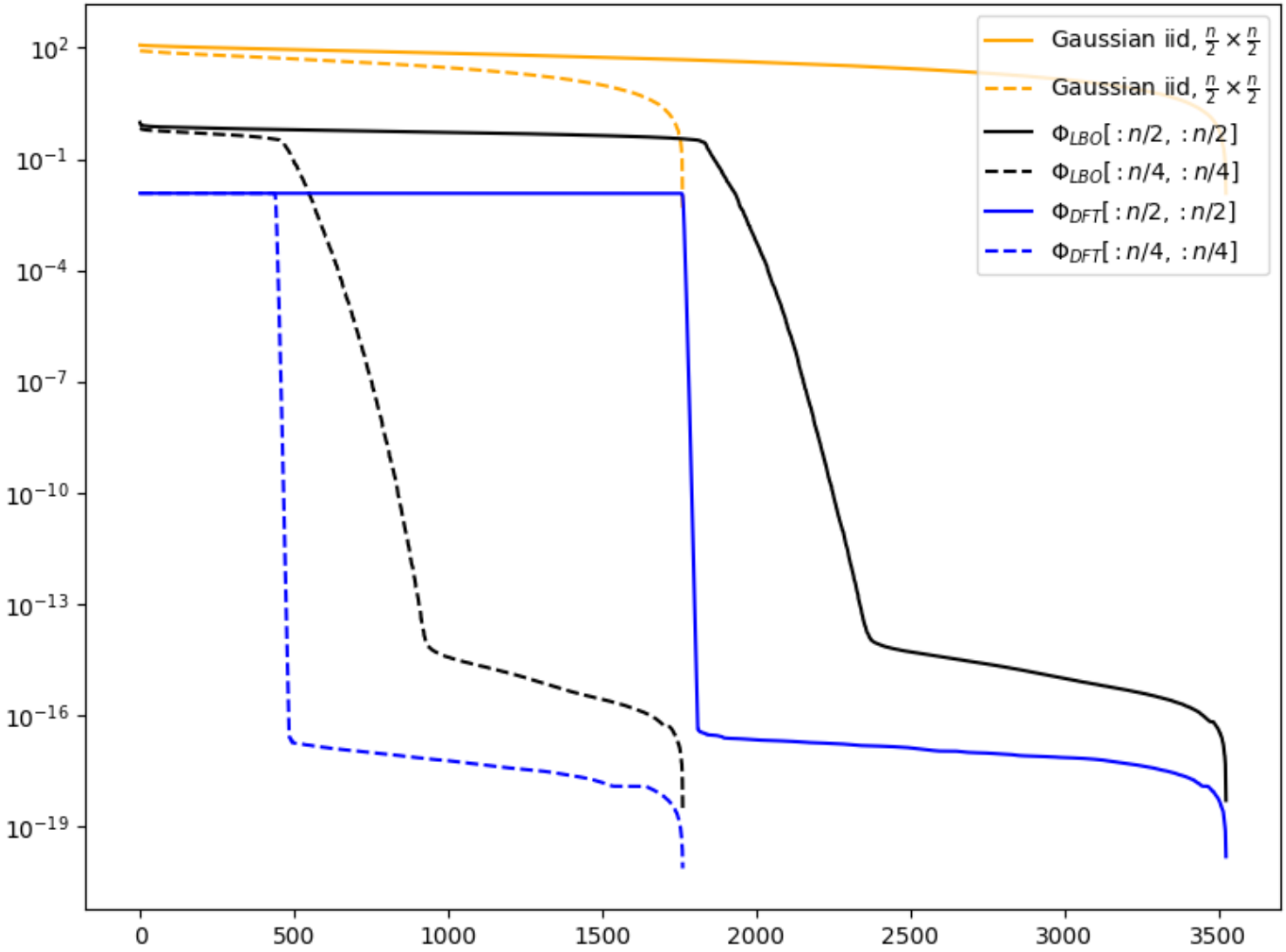
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **3,526** vertices)

Can we butterfly surface harmonics?

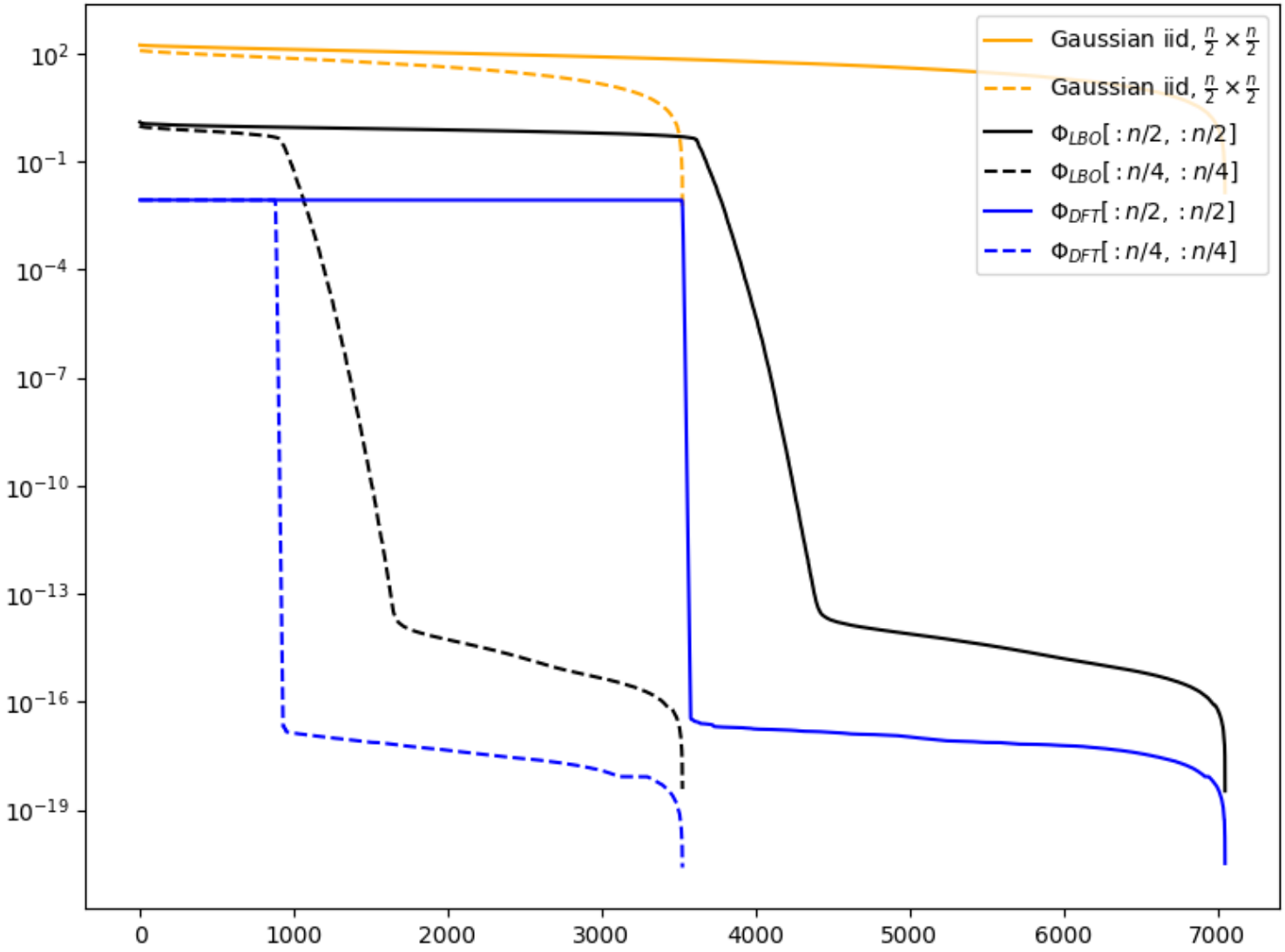
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **7,051** vertices)

Can we butterfly surface harmonics?

Surface harmonics are *weakly* butterfliable



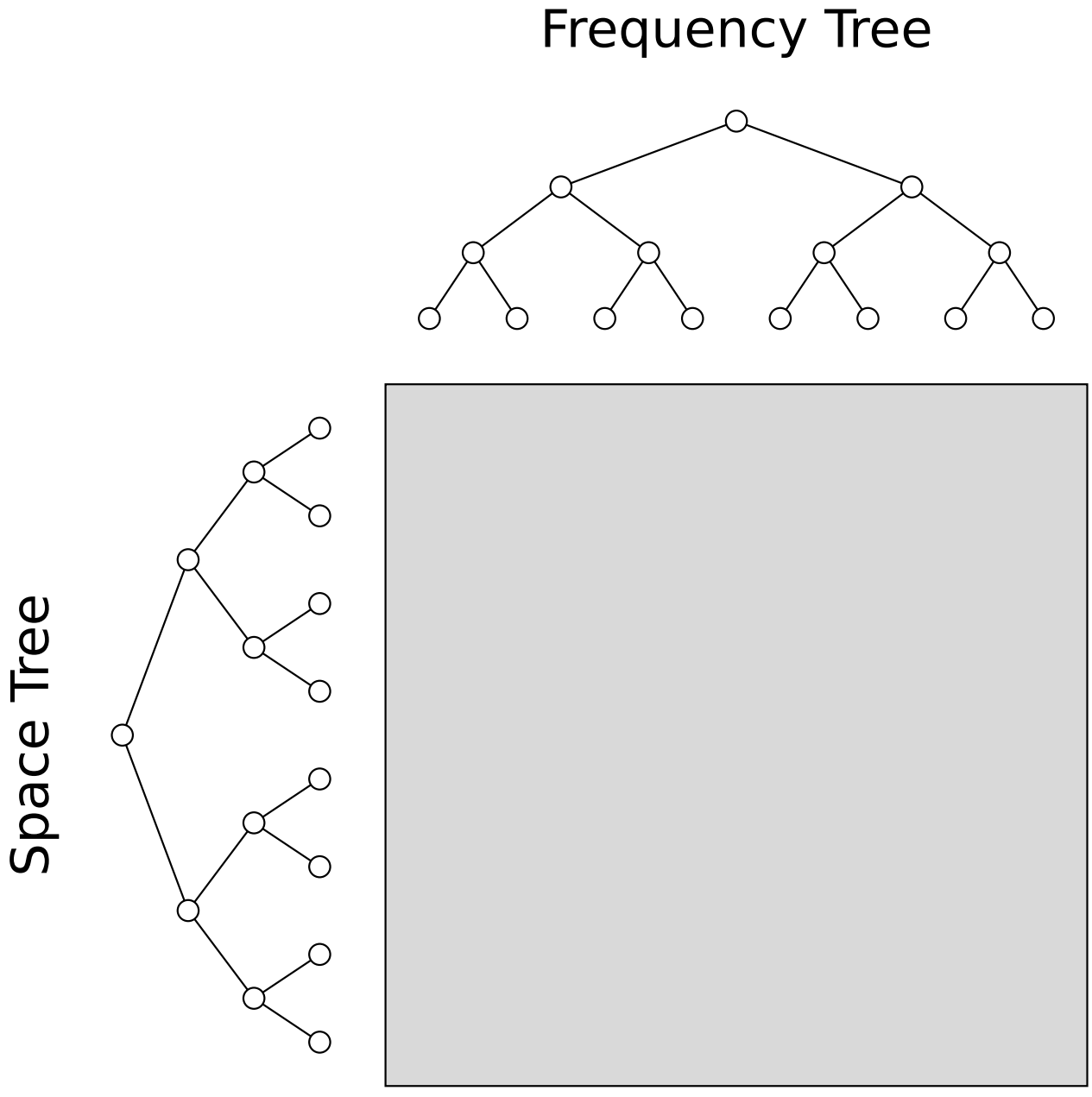
(“Asian dragon” mesh w/ **14,101** vertices)

Can we butterfly surface harmonics?

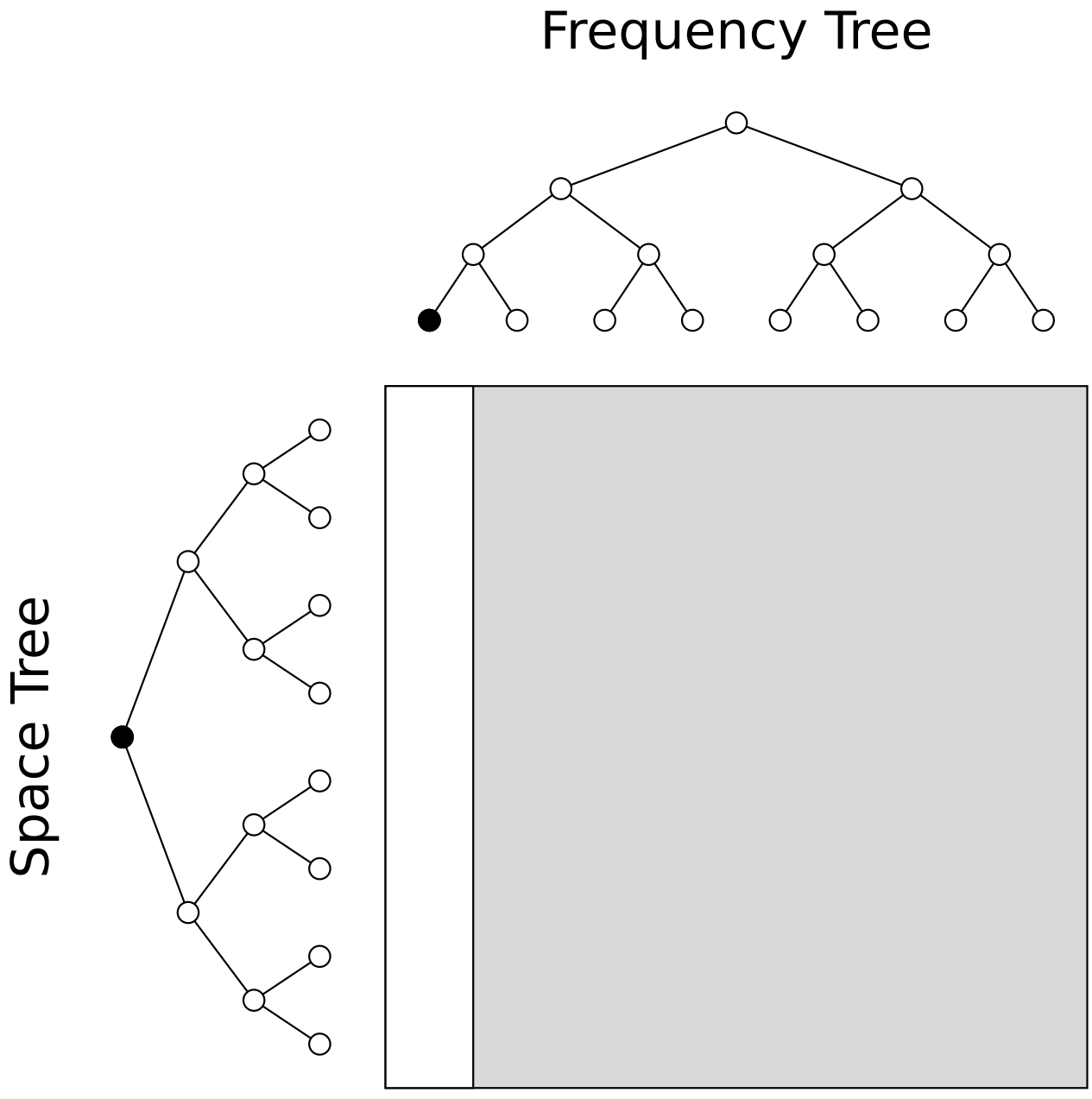
Yes, but. . .

- It will have worse constants than the FFT.
- Computing lots of eigenvectors requires a ton of memory.
- Weyl's law indicates that the butterfly will be nonuniform (more modes per frequency for higher frequencies).
- FEM approximation is poor at higher frequencies.
- We will need some fancy new algorithms to fight these problems. . .
- **Nevertheless**, many applications really only need $O(1)$ modes. . .

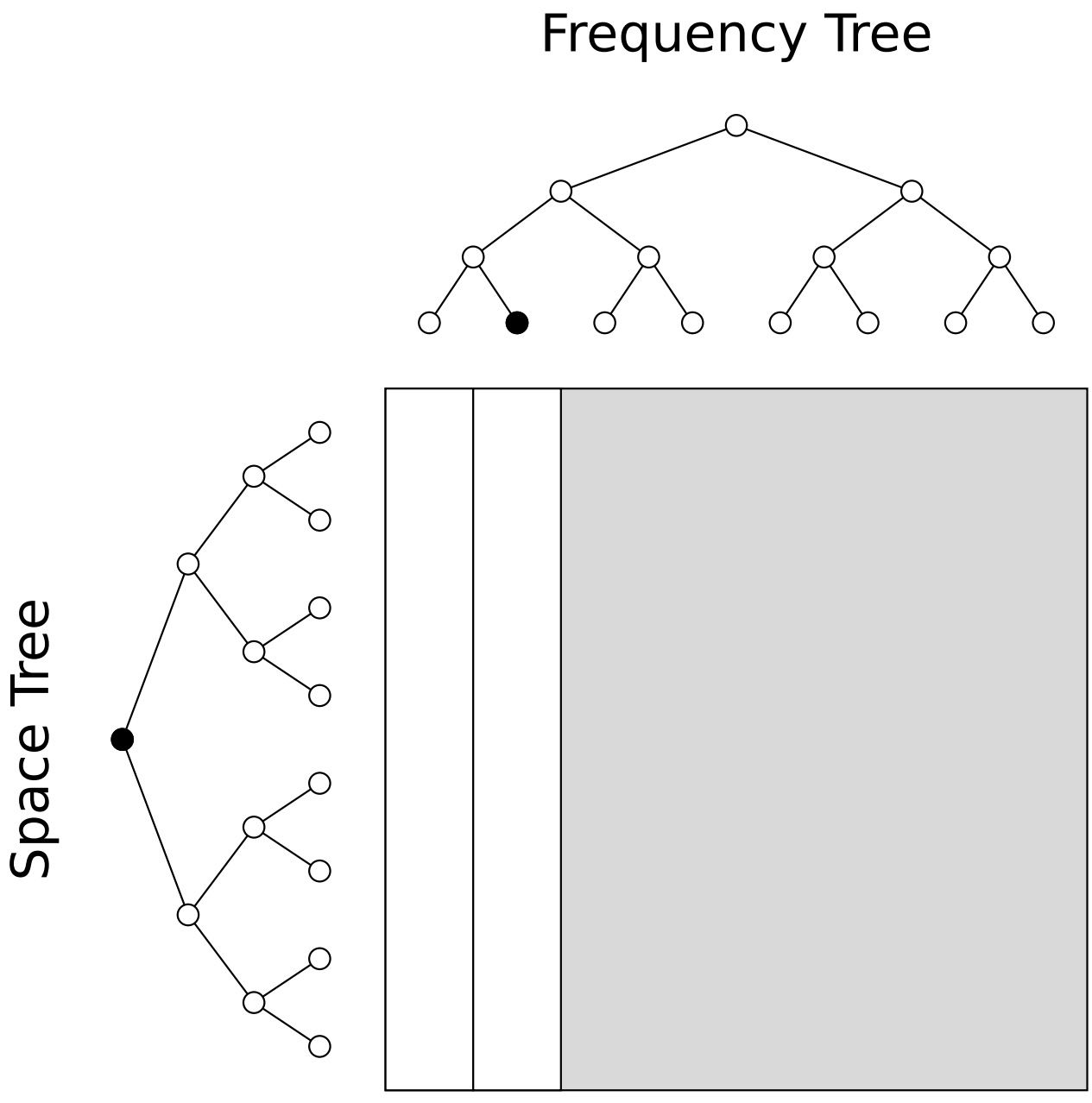
Streaming a butterfly factorization



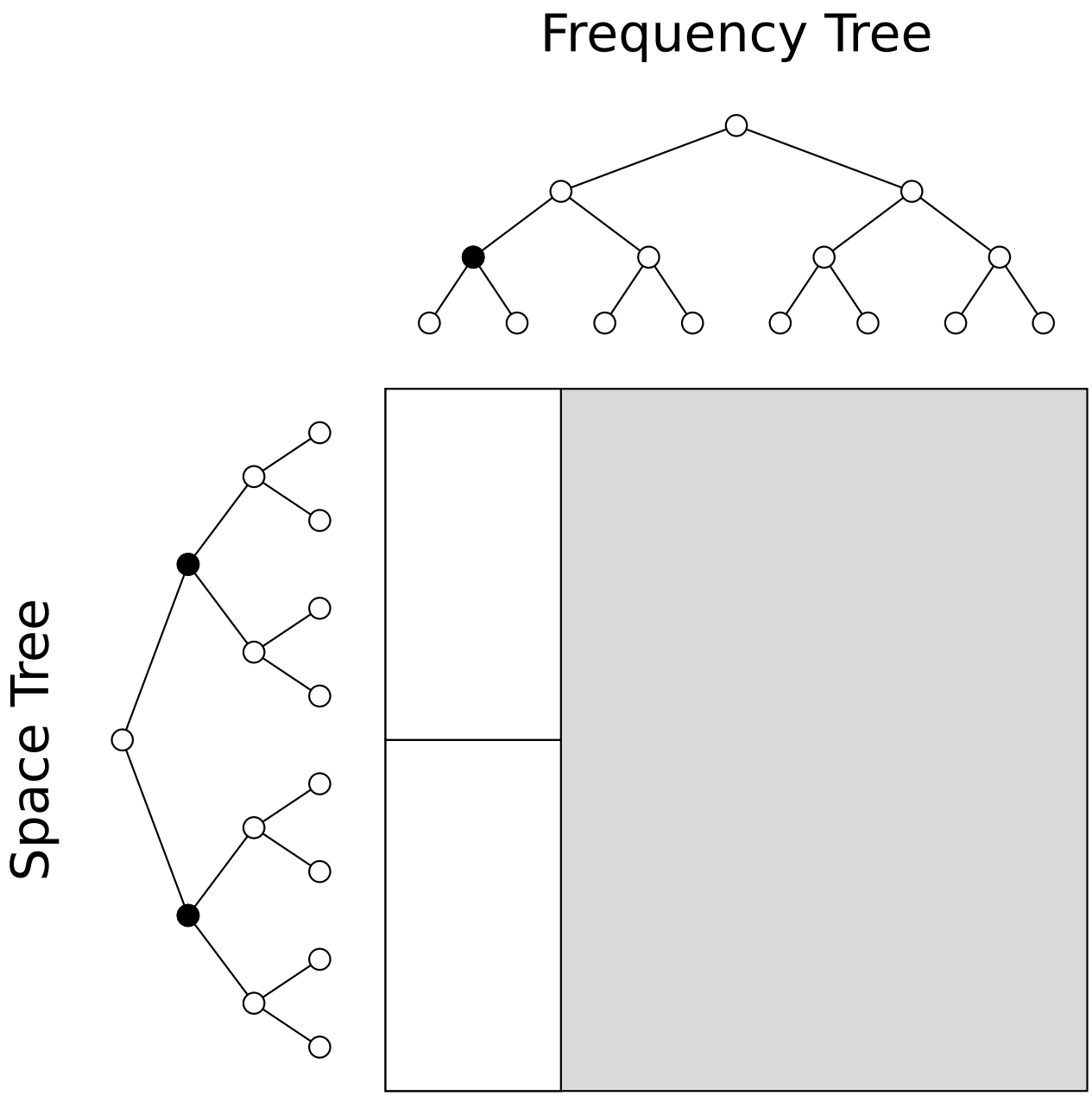
Streaming a butterfly factorization



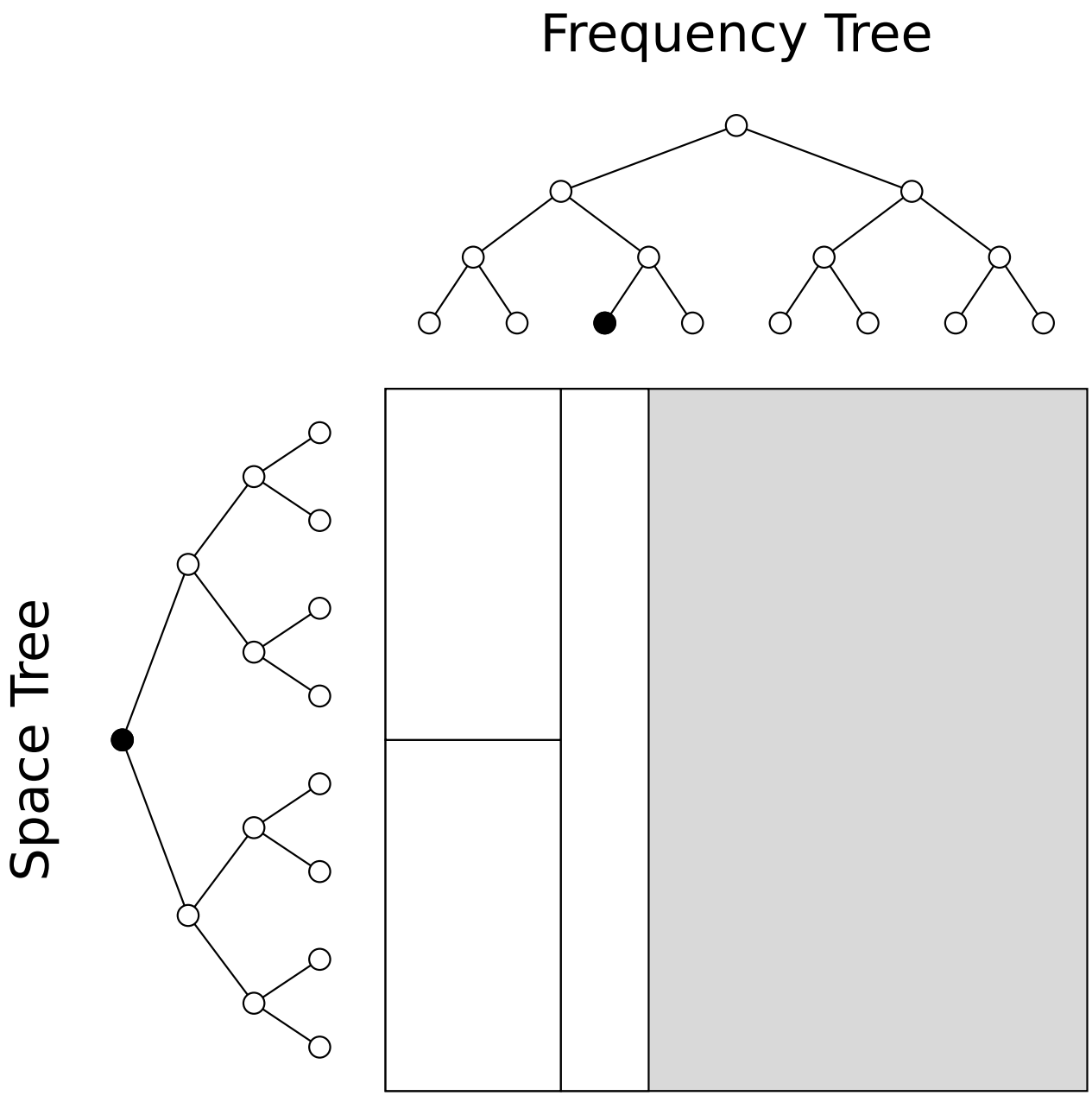
Streaming a butterfly factorization



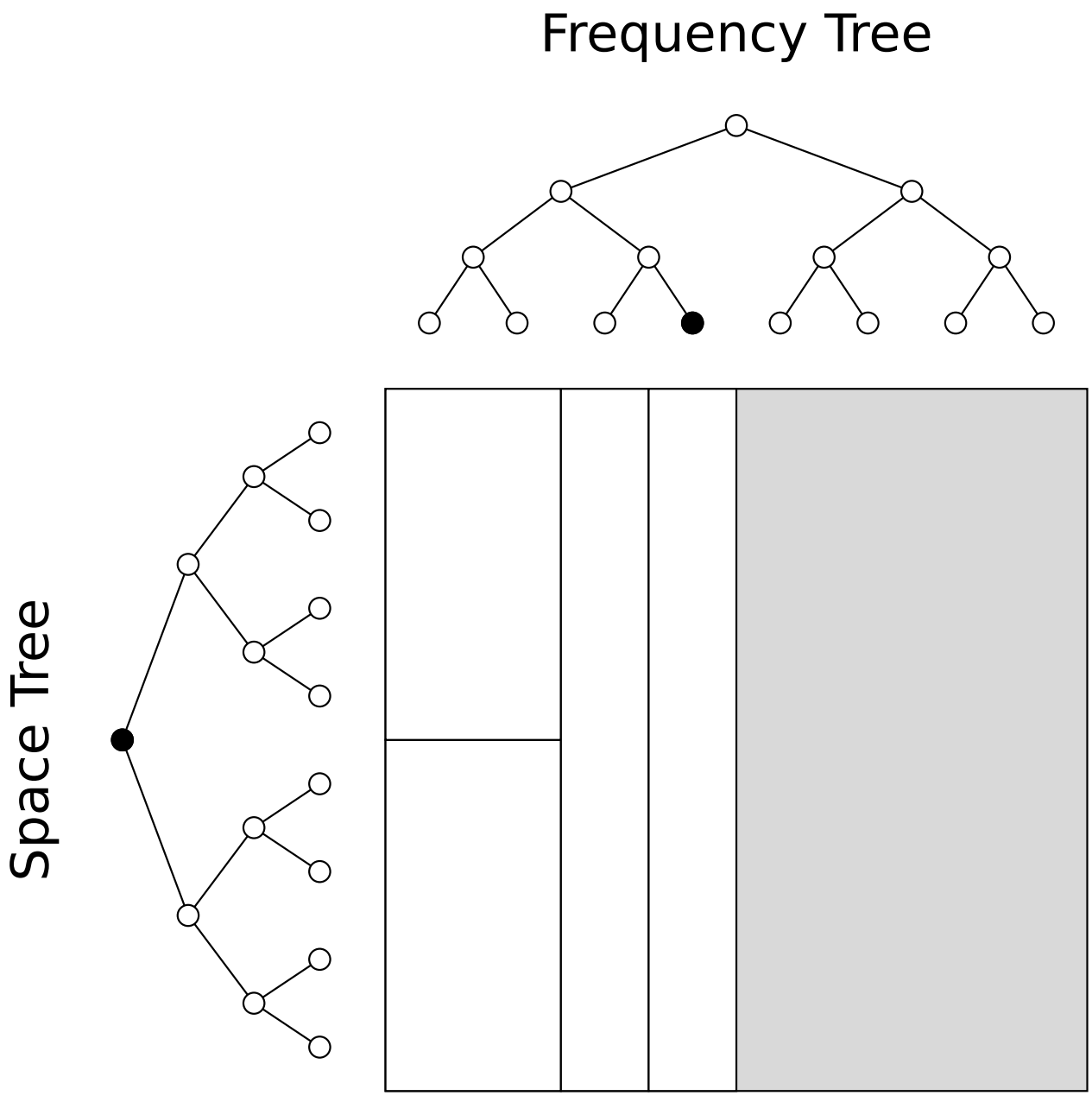
Streaming a butterfly factorization



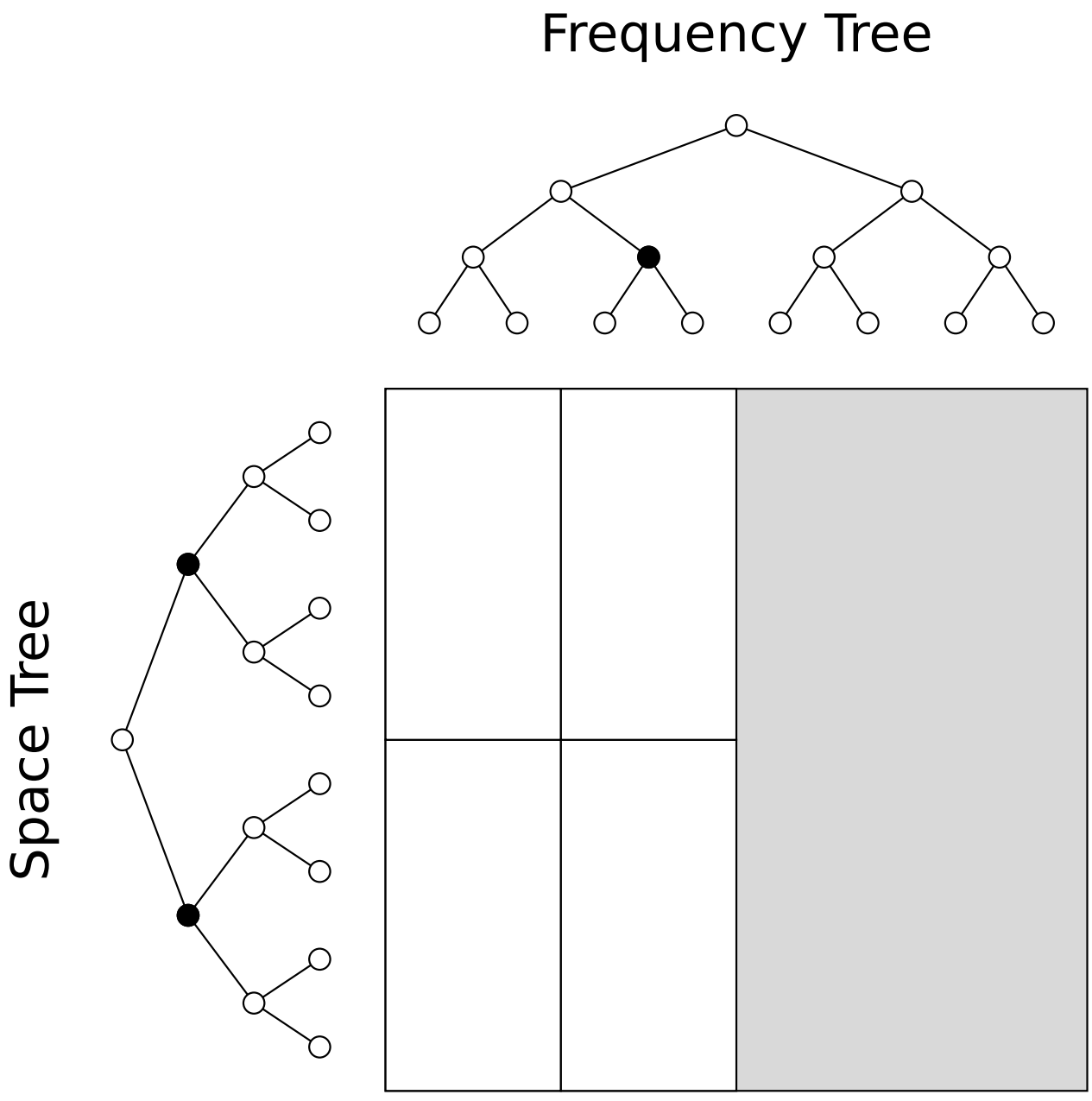
Streaming a butterfly factorization



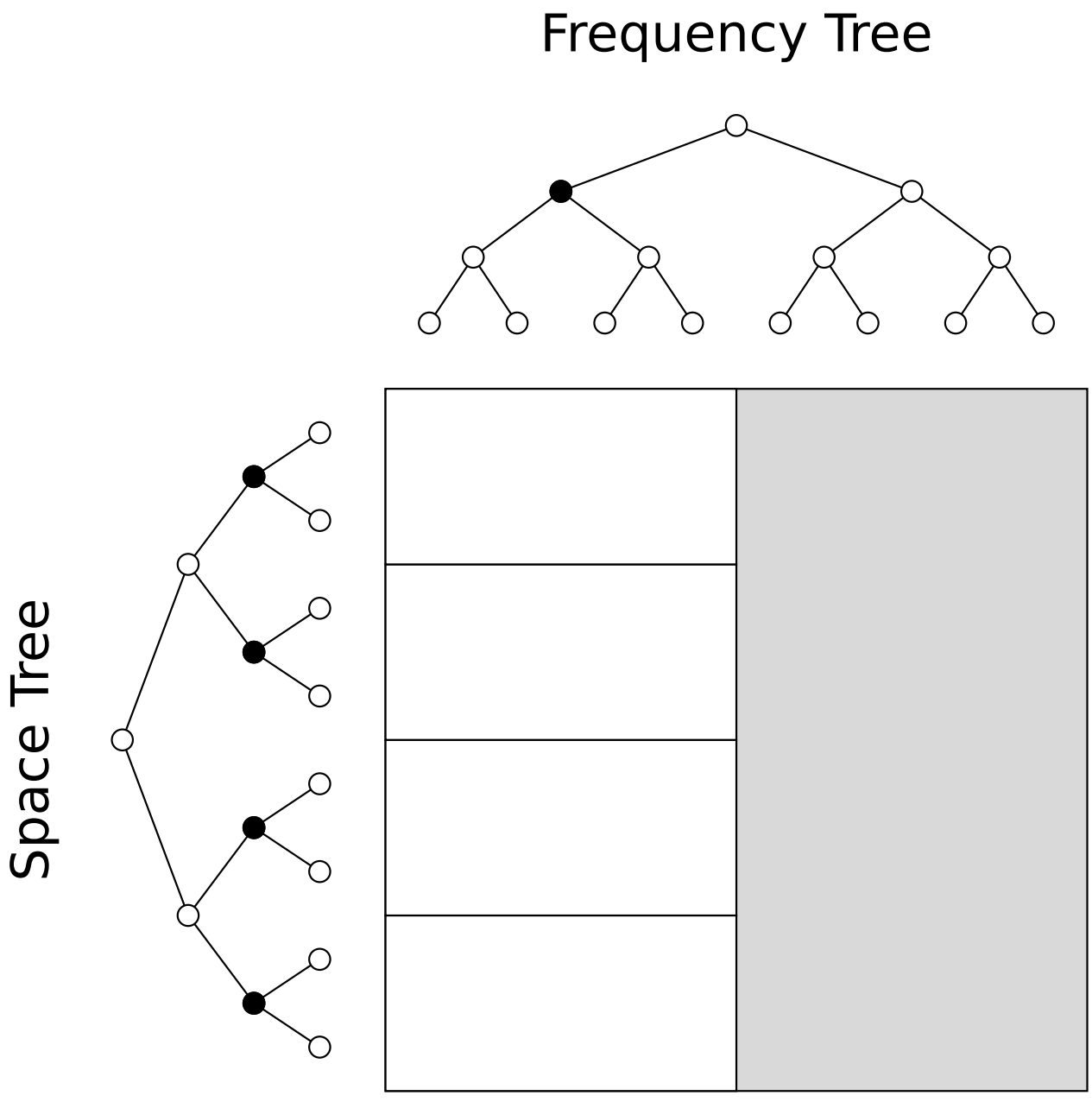
Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming a butterfly factorization

- Cheap to compute $\lambda_{\min}$ and $\lambda_{\max}$.
- Set up grid of eigenvalues:

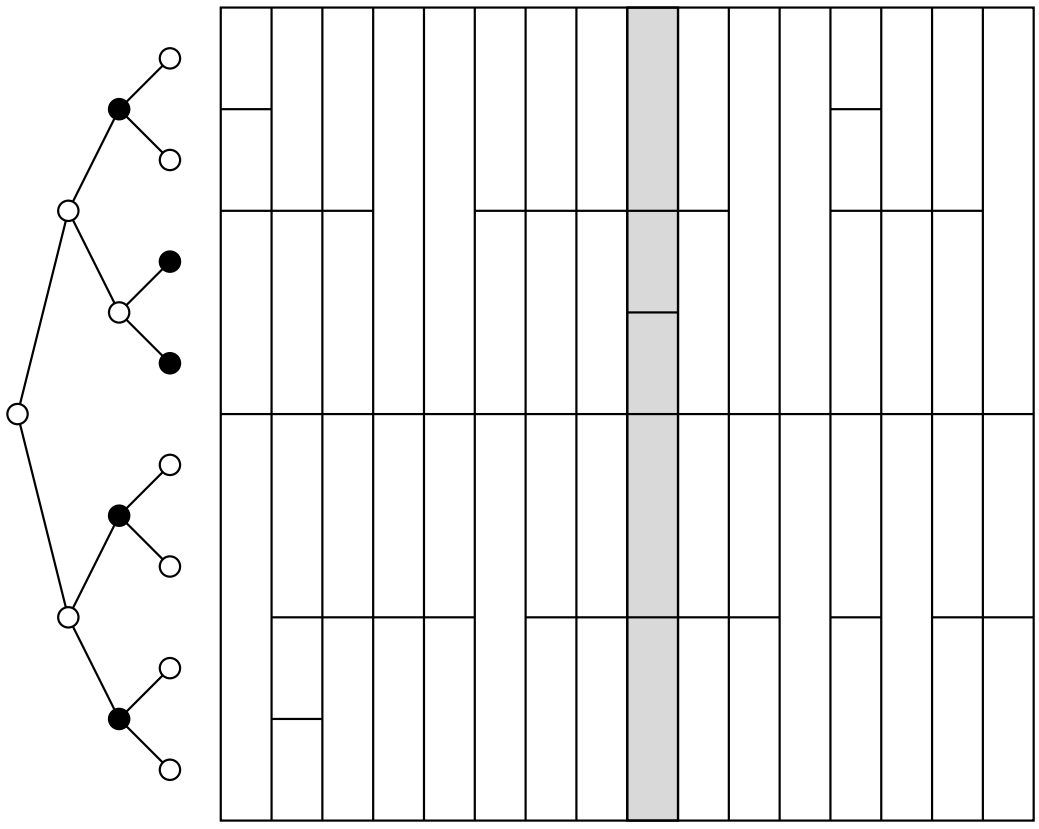
$$\tilde{\lambda}_0 = \lambda_{\min} < \tilde{\lambda}_1 < \cdots < \tilde{\lambda}_{N-1} < \tilde{\lambda}_N = \lambda_{\max}$$

  Compute all pairs within each band $[\tilde{\lambda}_{k-1}, \tilde{\lambda}_k)$.
- To do this efficiently, we use **spectrum covering**. Old school technique not used much any more. Ask me about it if you're interested.

Compute p eigenpairs in roughly $O(np)$ time. Can be used to stream bands one at a time.

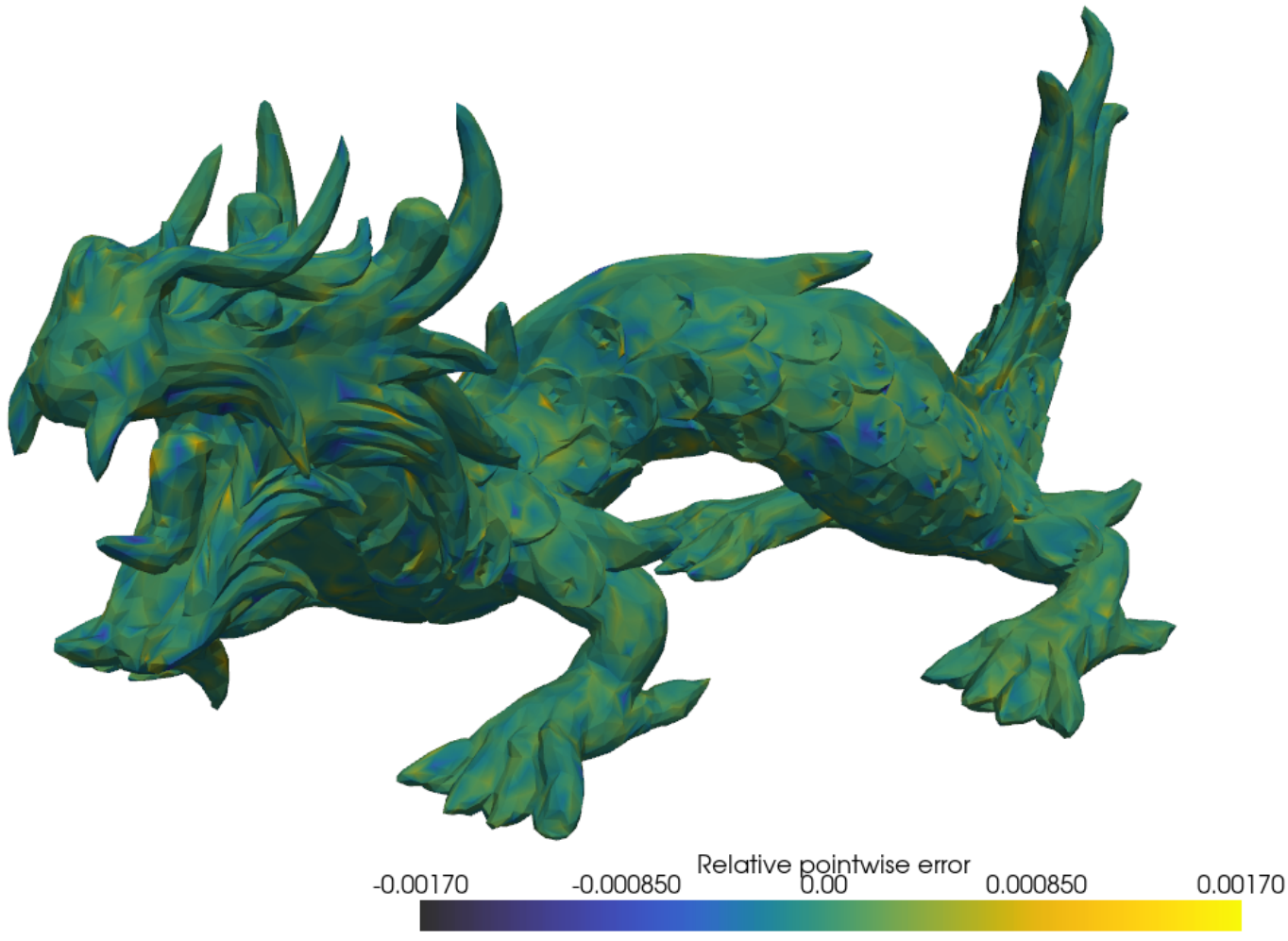
Adaptive butterfly factorization

For each column, find the highest cut through the spatial tree such that each subblock can be truncated with the desired precision.



Numerical results

For low-rank factorizations, use truncated SVD, keeping singular vectors where $\sigma_k \geq \epsilon \sigma_1$.



Numerical results

Results for adaptive BF with relative error $\epsilon = 10^{-3}$, computing lowest $\frac{1}{16}$th of spectrum.

Verts	Eigs	Orig	BF	Ratio	Order
1,764	110	1.5 MB	0.9 MB	1.67	
3,526	220	5.9 MB	3.0 MB	1.97	1.74
7,051	440	23.7 MB	9.7 MB	2.44	1.69
14.1K	881	94.8 MB	30.2 MB	3.14	1.64
28.2K	1,762	379.1 MB	92.8 MB	4.09	1.62
56.4K	3,524	1.5 GB	278.7 MB	5.44	**1.59**
113K	7,049	6.0 GB	863.7 MB	7.02	1.63
225K	14.1K	**24.2 GB**	**2.5 GB**	9.42	**1.58**

Complexity possibly $O(n^{\log_2(3)})$ ($\log_2(3) \approx 1.585\ldots$)?

Gaussian random fields on surfaces

Let Z be mean zero Gaussian random field on M.

For pairs of points $x, y \in M$, have:

- $\mathbb{E}Z(x) = 0$,
- $\mathrm{Cov}(Z(x), Z(y)) = \mathbb{E}Z(x)Z(y) = k(x, y)$,

where k is a positive definite kernel function.

If $k(x, y) = k(x - y)$, then Z has the K-L expansion:

$$Z(x) = \sum_{m=0}^{\infty} \sqrt{\gamma(\lambda_m)}\phi_m(x)W_m,$$

where $W_m \overset{\text{iid}}{\sim} \mathcal{N}(0, 1)$, γ is the spectral density, and ϕ_m is the mth continuous surface harmonic.

Approximating the covariance

Discrete K-L expansion using the FEM harmonics is:

$$Z(x_j) = \sum_{i=1}^{n} \sqrt{\gamma(\lambda_i)}\phi_i(x_j)W_i = D^{1/2}\Phi W.$$

The covariance is:

$$\Sigma = \mathrm{Cov}(Z) = \Phi D \Phi^{\mathsf{T}} = \sum_{i=1}^{n} \gamma(\lambda_i)\phi_i\phi_i^{\mathsf{T}}.$$

Error in discrete covariance using first k terms is:

$$\left\| \Sigma - \sum_{i=1}^{k} \gamma(\lambda_i)\phi_i\phi_i^{\mathsf{T}} \right\|^2 = \sum_{i=k+1}^{n} \gamma(\lambda_i)^2$$

Truncation error and Weyl's law

Theorem (Weyl 1911)

Let M be a compact d-dimensional manifold and $N(\lambda)$ the number of eigenvalues of Δ_M less than or equal to λ. Then:

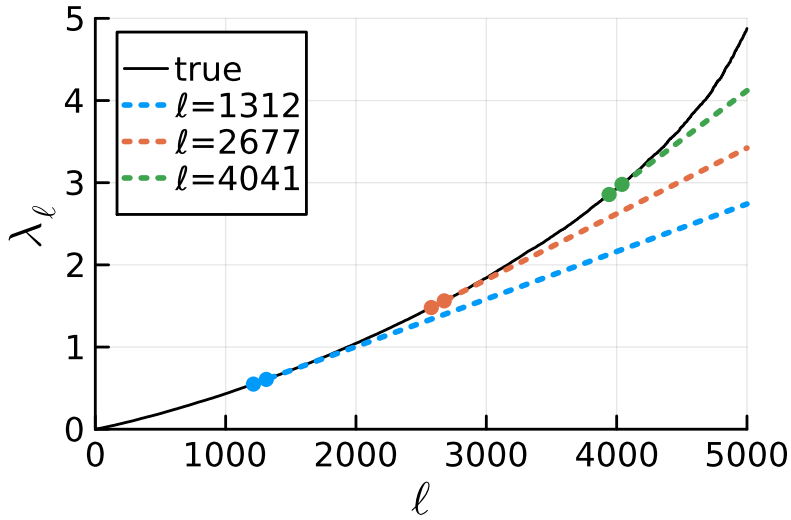
$$\lim_{\lambda \to \infty} \frac{N(\lambda)}{\lambda^{d/2}} = \textit{constant dependent on } M.$$

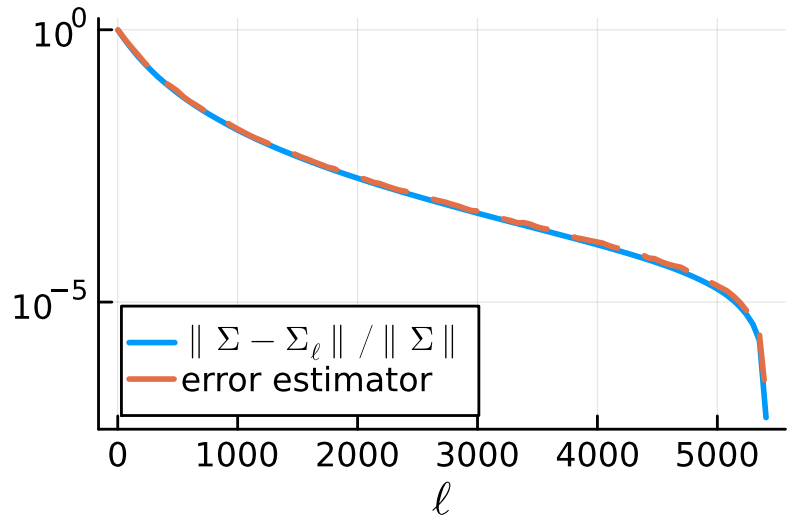
Idea: while streaming the butterfly factorization, extrapolate λ_i using Weyl's law (i.e. do a linear fit) and use this to estimate the error of the K-L expansion. Stop when a threshold is reached.

Result: fast sampling from GRFs, fast covariance matrix apply.

Numerical results

Extrapolation and error estimator (very accurate!):





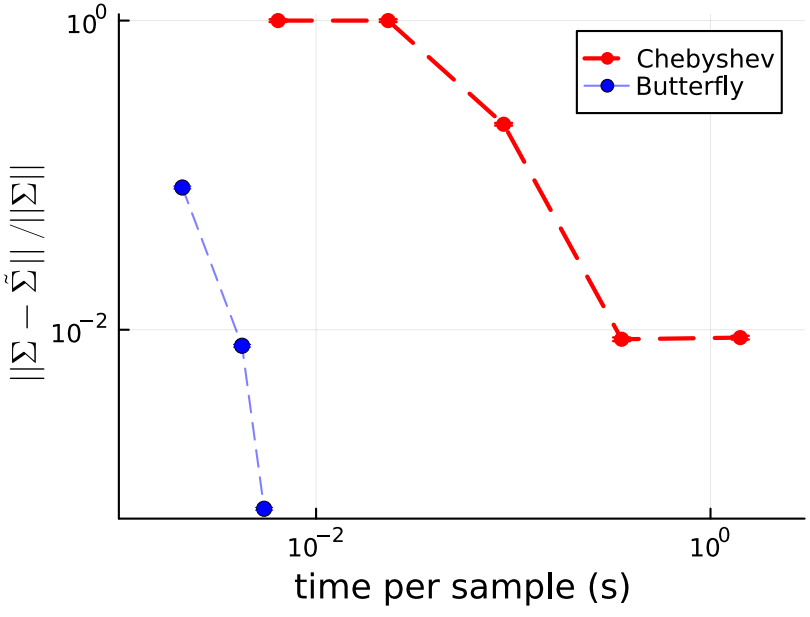
Compare speed of sampling using butterfly factorization against method of Lang and Pereira[1]. Basically, approximate γ with a Chebyshev series g, sample by computing:

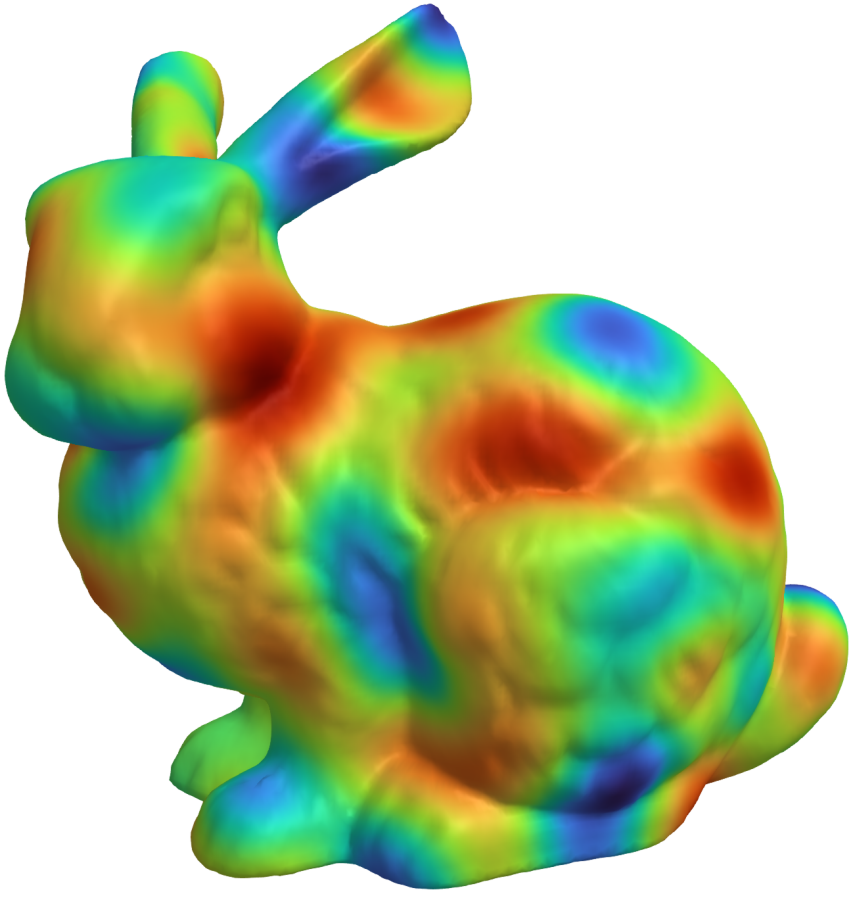
$$Z = \sqrt{M}^{-\mathsf{T}} g(\sqrt{M}^{-1} L \sqrt{M}^{-\mathsf{T}}) W.$$

[1] "GalerkinChebyshev approximation of Gaussian random fields on compact Riemannian manifolds," arXiv:2107.02667, 2021.

Numerical results

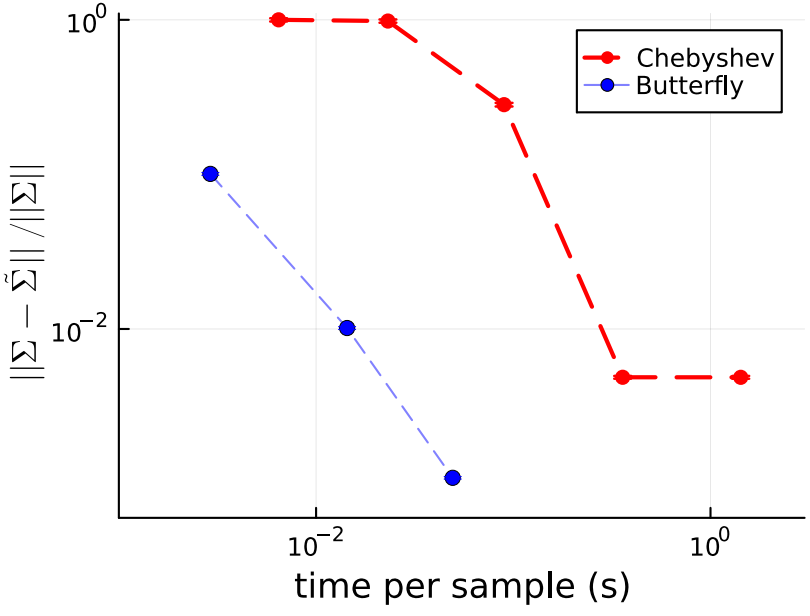
Stanford bunny (34,817 vertices), $\gamma(\lambda) = \exp(-\lambda^2/\alpha)$, where $\alpha = 10,000$.

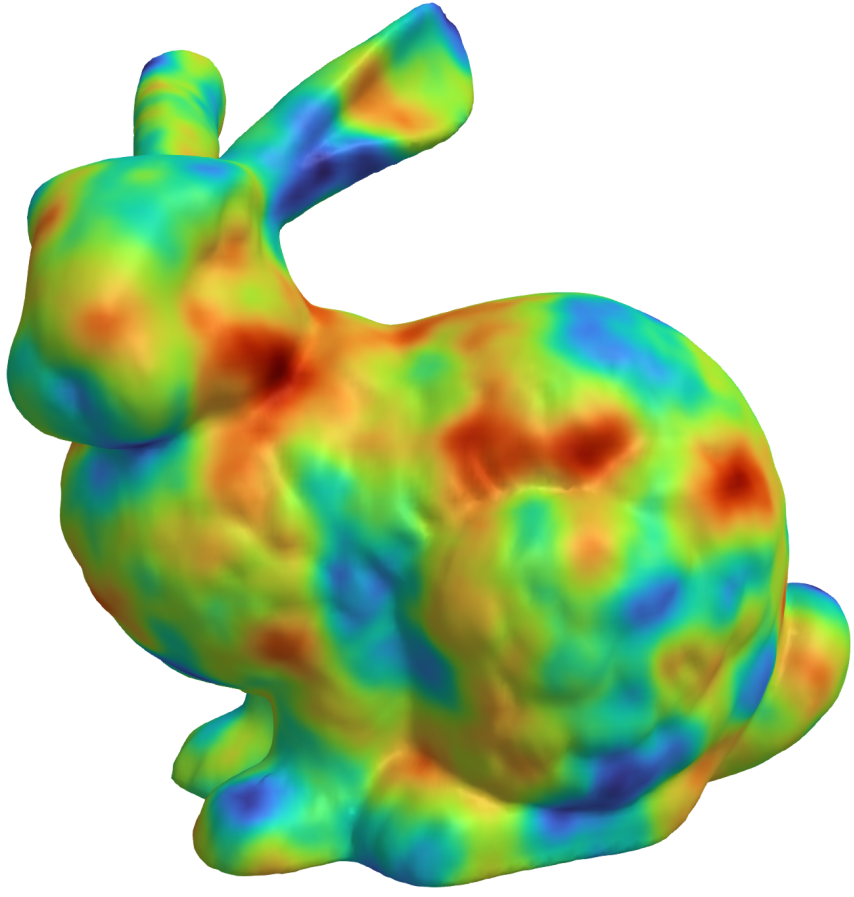




Numerical results

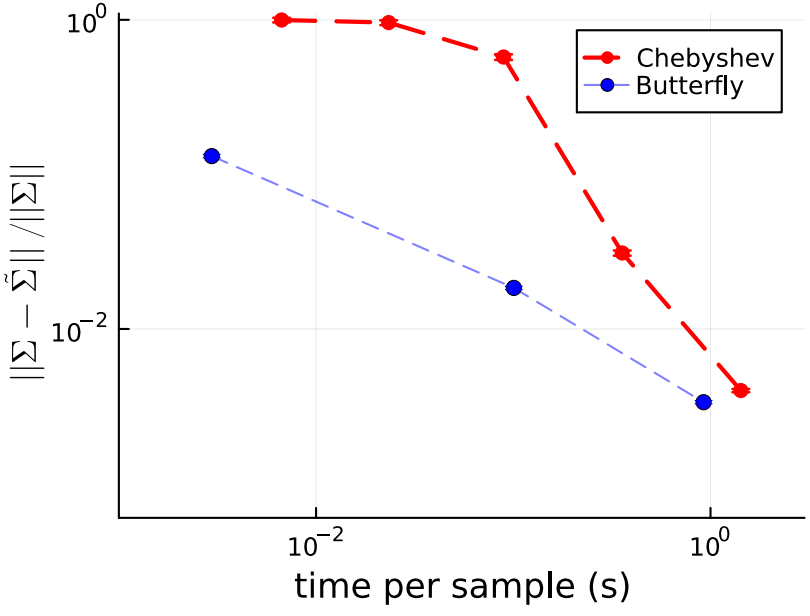
Stanford bunny (34,817 vertices), $\gamma(\lambda) = (\alpha^2 + \lambda)^{-\frac{\nu}{4}-\frac{1}{2}}$, where $\nu = 4$, $\alpha = 10$.

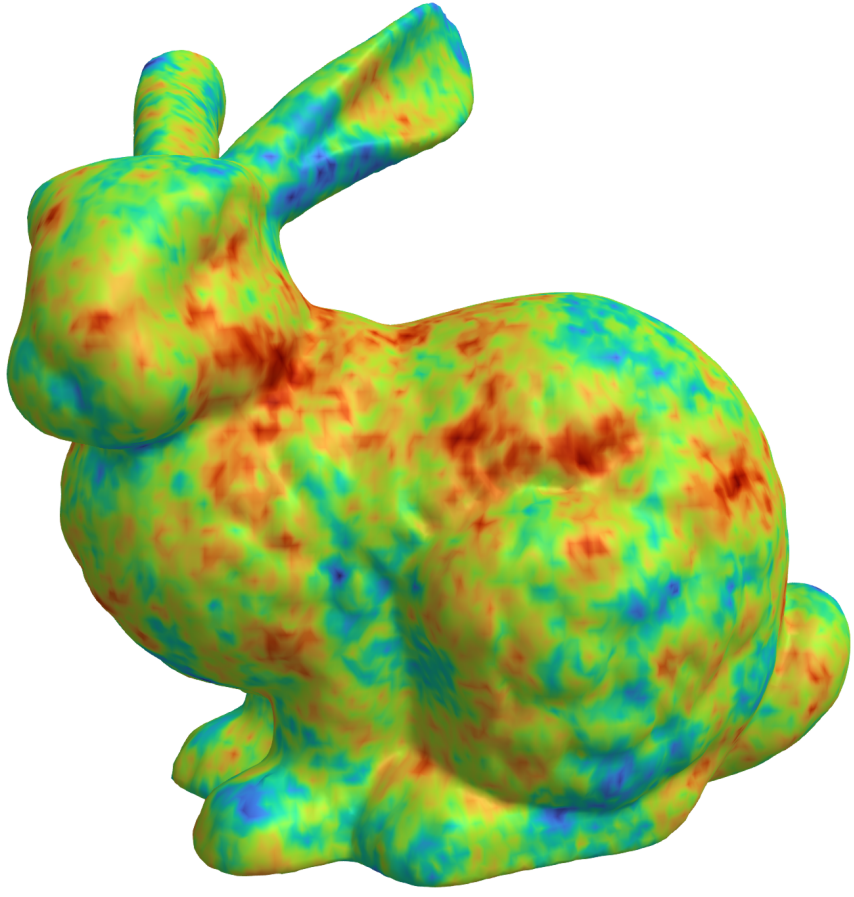




Numerical results

Stanford bunny (34,817 vertices), $\gamma(\lambda) = (\alpha^2 + \lambda)^{-\frac{\nu}{4}-\frac{1}{2}}$, where $\nu = 1/2$, $\alpha = 10/3$.





Thanks!

Samuel F Potter, Paul G Beckman, Mike O'Neil, "Butterfly-accelerated Gaussian random fields on manifolds", *in preparation*.

(Expect this on arXiv soon. . .)